

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống chấm thi trắc nghiệm thông minh
tích hợp nhận dạng OMR và quản lý đào tạo**

NGUYỄN VĂN ABC

nguyenvanabc@sis.hust.edu.vn

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống chấm thi trắc nghiệm thông minh
tích hợp nhận dạng OMR và quản lý đào tạo

NGUYỄN VĂN ABC

nguyenvanabc@sis.hust.edu.vn

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành tới PGS. TS. Phạm Văn ABC, giảng viên hướng dẫn của đề án, người đã tận tình chỉ bảo, định hướng nghiên cứu và đóng góp những nhận xét quý báu giúp em hoàn thiện đề án này trong suốt thời gian qua.

Em cũng xin trân trọng cảm ơn các thầy cô trong Khoa Khoa học máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã trang bị cho em nền tảng kiến thức chuyên môn và tạo điều kiện để em thực hiện đề án. Em cảm ơn các bạn bè cùng khóa đã thường xuyên thảo luận, chia sẻ kinh nghiệm và hỗ trợ em trong quá trình xây dựng cũng như kiểm thử hệ thống Smart Grading.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc tới gia đình và bạn bè đã luôn động viên, khuyến khích em vượt qua những khó khăn trong quá trình học tập và thực hiện đề án tốt nghiệp. Em cũng tự cảm ơn chính mình đã kiên trì và nỗ lực để hoàn thành đề án này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh các kỳ thi trắc nghiệm quy mô lớn ngày càng phổ biến tại các trường phổ thông và đại học, việc chấm thi thủ công trên phiếu điền tay vẫn tồn tại nhiều hạn chế như tốn thời gian, dễ sai sót, thiếu tính minh bạch và khó khăn trong việc thống kê, phân tích kết quả. Một số giải pháp chấm thi tự động hiện có thường phụ thuộc vào phần cứng máy quét chuyên dụng đắt tiền, hoặc chỉ giải quyết được một phần quy trình như tạo đề hay nhận dạng phiếu trả lời mà chưa cung cấp một hệ thống tích hợp đầy đủ từ quản lý ngân hàng câu hỏi, trộn đề, chấm thi bằng nhận dạng OMR trên thiết bị di động, đến xử lý phúc khảo và báo cáo thống kê.

Đồ án lựa chọn hướng tiếp cận xây dựng một hệ thống chấm thi trắc nghiệm thông minh đa nền tảng, tích hợp nhận dạng OMR trên thiết bị di động, tự động tạo nhiều phiên bản đề thi từ ngân hàng câu hỏi, hỗ trợ quy trình phúc khảo minh bạch và cung cấp các báo cáo thống kê chi tiết cho giáo viên và quản trị viên. Hệ thống sử dụng kiến trúc ba tầng gồm backend Node.js/Express với cơ sở dữ liệu MongoDB, ứng dụng di động Flutter cho thầy cô sử dụng máy ảnh thiết bị để quét và chấm bài, giao diện web React cho quản lý và thống kê, kết hợp gói LaTeX AMC (Auto-Multiple-Choice) để tự động sinh đề thi và phiếu trả lời.

Kết quả chính của đồ án là hệ thống Smart Grading đã được xây dựng hoàn chỉnh với các chức năng:

- quản lý trường học, lớp học, giáo viên và học sinh;
- quản lý ngân hàng câu hỏi và đề thi có hỗ trợ tạo câu hỏi bằng trí tuệ nhân tạo (Gemini, OpenAI, Claude);
- tự động trộn đề thi thành nhiều phiên bản khác nhau thông qua AMC và tạo phiếu OMR tương ứng;
- chấm thi tự động bằng nhận dạng OMR trên thiết bị di động với độ chính xác cao, có khả năng phát hiện lỗi tô nhầm, tô nhẹ và hỗ trợ chỉnh sửa thủ công;
- quy trình tạo đơn phúc khảo và phản hồi phúc khảo minh bạch với thông báo thời gian thực;
- hệ thống báo cáo và thống kê chi tiết cho từng bài kiểm tra.

Hệ thống đã được triển khai thử nghiệm tại hai lớp học với khoảng 60 học sinh trong bốn tuần, xử lý thành công 5 bài kiểm tra và khoảng 300 bài làm, đạt độ chính xác OMR trung bình 96,2% và thời gian phản hồi API trung bình 180ms.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)
NGUYỄN VĂN ABC

ABSTRACT

In the context of large-scale multiple-choice examinations becoming increasingly common in secondary schools and universities, manual grading of paper-based answer sheets still suffers from many limitations including time consumption, proneness to errors, lack of transparency, and difficulty in statistical analysis. Existing automated grading solutions often rely on expensive specialized scanning hardware or only address a partial workflow such as question generation or answer sheet recognition, without providing an integrated system covering the entire process from question bank management and exam versioning to OMR-based grading on mobile devices, transparent appeal handling, and comprehensive reporting.

This thesis adopts the approach of building a multi-platform smart multiple-choice grading system that integrates Optical Mark Recognition (OMR) on mobile devices, automatically generates multiple exam versions from a shared question bank, supports a transparent appeal workflow, and provides detailed statistical reports for teachers and administrators. The system follows a three-tier architecture comprising a Node.js/Express backend with MongoDB database, a Flutter mobile application that leverages the device camera for scanning and grading, a React web dashboard for administration and analytics, combined with the AMC (Auto-Multiple-Choice) LaTeX package for automatic exam paper and answer sheet generation.

The main contribution of the thesis is the complete Smart Grading system with the following capabilities:

- management of schools, classes, teachers, and students;
- question bank and exam management with AI-assisted question generation using Gemini, OpenAI, and Claude;
- automatic generation of multiple exam versions and corresponding OMR answer sheets through AMC;
- mobile OMR-based automatic grading with high accuracy, including detection of double marks and light marks, with manual override capability;
- a transparent appeal submission and response workflow with real-time notifications;
- detailed per-exam reports and analytics with system-assisted AI tutor interaction;

- a cross-platform mobile application that enables grading directly in the classroom without the need for specialized scanning equipment.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Tổng quan chương	1
1.2 Đặt vấn đề.....	1
1.3 Mục tiêu và phạm vi đề tài.....	1
1.4 Định hướng giải pháp.....	2
1.5 Bố cục đồ án	3
1.6 Kết chương.....	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Tổng quan chương	5
2.2 Khảo sát hiện trạng	5
2.3 Tổng quan chức năng	6
2.3.1 Biểu đồ use case phân rã Đăng ký/Đăng nhập.....	8
2.3.2 Biểu đồ use case phân rã Quản lý trường học	8
2.3.3 Biểu đồ use case phân rã Quản lý giáo viên	8
2.3.4 Biểu đồ use case phân rã Quản lý học sinh	8
2.3.5 Biểu đồ use case phân rã Tạo câu hỏi, đề thi	9
2.3.6 Biểu đồ use case phân rã Tạo đề thi, trộn các câu cho nhiều đề thi ..	9
2.3.7 Biểu đồ use case phân rã Quản lý các bài thi của học sinh.....	9
2.3.8 Biểu đồ use case phân rã Chấm thi bằng ứng dụng	10
2.3.9 Biểu đồ use case phân rã Tạo đơn phúc khảo	10
2.3.10 Biểu đồ use case phân rã Phản hồi phúc khảo	10
2.3.11 Biểu đồ use case phân rã Xem bài thi và điểm số.....	11
2.3.12 Biểu đồ use case phân rã Tạo báo cáo cho bài kiểm tra	11
2.3.13 Quy trình nghiệp vụ.....	11

2.4 Đặc tả chức năng	12
2.4.1 Đặc tả use case Đăng ký/Đăng nhập	12
2.4.2 Đặc tả use case Tạo đề thi, trộn các câu cho nhiều đề thi	13
2.4.3 Đặc tả use case Chấm thi bằng ứng dụng	14
2.4.4 Đặc tả use case Tạo đơn phúc khảo	15
2.4.5 Đặc tả use case Phản hồi phúc khảo	16
2.4.6 Đặc tả use case Tạo báo cáo cho bài kiểm tra	17
2.5 Yêu cầu phi chức năng	18
2.6 Kết chương.....	19
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	20
3.1 Tổng quan chương	20
3.2 Kỹ thuật nhận dạng OMR và xử lý ảnh số	20
3.3 Kỹ thuật tạo đề nhiều phiên bản với AMC	21
3.4 Công nghệ phát triển backend	22
3.5 Công nghệ phát triển giao diện web.....	22
3.6 Công nghệ phát triển ứng dụng di động	23
3.7 Công nghệ trí tuệ nhân tạo tích hợp	24
3.8 Các công nghệ hỗ trợ khác.....	24
3.9 Kết chương.....	25
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	26
4.1 Tổng quan chương	26
4.2 Thiết kế kiến trúc.....	26
4.2.1 Lựa chọn kiến trúc phần mềm	26
4.2.2 Thiết kế tổng quan.....	26
4.2.3 Thiết kế chi tiết gói	28

4.3 Thiết kế chi tiết.....	29
4.3.1 Thiết kế giao diện	29
4.3.2 Thiết kế lớp	31
4.3.3 Thiết kế cơ sở dữ liệu	33
4.4 Xây dựng ứng dụng.....	34
4.4.1 Thư viện và công cụ sử dụng.....	34
4.4.2 Kết quả đạt được	36
4.4.3 Minh họa các chức năng chính	36
4.5 Kiểm thử.....	37
4.6 Triển khai	38
4.7 Kết chương.....	38
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	40
5.1 Tổng quan chương	40
5.2 Giải pháp tự động sinh đề thi nhiều phiên bản và phiếu OMR kèm tọa độ nhận dạng.....	40
5.2.1 Dẫn dắt bài toán.....	40
5.2.2 Giải pháp chi tiết.....	40
5.2.3 Kết quả đạt được	41
5.3 Giải pháp nhận dạng OMR trên thiết bị di động với khả năng phát hiện lỗi	41
5.3.1 Dẫn dắt bài toán.....	41
5.3.2 Giải pháp chi tiết.....	41
5.3.3 Kết quả đạt được	42
5.4 Giải pháp tích hợp trợ lý AI trong sinh câu hỏi và phản hồi phức khảo.....	43
5.4.1 Dẫn dắt bài toán.....	43
5.4.2 Giải pháp chi tiết.....	43
5.4.3 Kết quả đạt được	44

5.5 Giải pháp quản lý phiên làm việc và phân quyền theo vai trò trên đa nền tảng	44
5.5.1 Dẫn dắt bài toán.....	44
5.5.2 Giải pháp chi tiết.....	44
5.5.3 Kết quả đạt được	45
5.6 Giải pháp thống kê và phân tích kết quả bài kiểm tra.....	45
5.6.1 Dẫn dắt bài toán.....	45
5.6.2 Giải pháp chi tiết.....	45
5.6.3 Kết quả đạt được	45
5.7 Kết chương.....	46
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
6.1 Tổng quan chương	47
6.2 Kết luận	47
6.2.1 So sánh với các sản phẩm tương tự.....	47
6.2.2 Những việc đã làm được	47
6.2.3 Những việc chưa làm được.....	48
6.2.4 Bài học kinh nghiệm	49
6.3 Hướng phát triển.....	49
6.3.1 Hoàn thiện các chức năng hiện có.....	49
6.3.2 Mở rộng tính năng mới	49
6.3.3 Nâng cấp công nghệ	50
6.4 Kết chương.....	50
TÀI LIỆU THAM KHẢO.....	54
PHỤ LỤC.....	56
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	56
A.1 Ngành học.....	57

A.2 Đánh dấu (bullet) và đánh số (numering)	57
A.3 Cách thêm bảng	58
A.4 Chèn hình ảnh	58
A.5 Tài liệu tham khảo	59
A.6 Cách viết phương trình và công thức toán học.....	59
A.7 Qui cách đóng quyển.....	59
B. ĐẶC TẢ USE CASE.....	61
B.1 Đặc tả use case Quản lý trường học.....	61
B.2 Đặc tả use case Quản lý giáo viên.....	62
B.3 Đặc tả use case Quản lý học sinh	62
B.4 Đặc tả use case Quản lý bài thi của học sinh	63
B.5 Đặc tả use case Xem bài thi và điểm số	64
B.6 Đặc tả use case Tạo câu hỏi, đề thi.....	65
B.7 Đặc tả use case Đăng ký/Đăng nhập	65

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống Smart Grading	7
Hình 4.1	Biểu đồ gói tổng quan của hệ thống Smart Grading	27
Hình 4.2	Ví dụ thiết kế gói biểu đồ lớp	28
Hình 4.3	Minh họa mockup giao diện dashboard trên web	30
Hình 4.4	Minh họa mockup giao diện quét phiếu trên di động	31
Hình A.1	Internet vạn vật	58
Hình A.2	Quy cách đóng quyển đồ án	60
Hình A.3	Quy cách đóng quyển đồ án	60

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh tính năng giữa các giải pháp chấm thi hiện có	6
Bảng 4.1	Danh sách thư viện và công cụ sử dụng trong đồ án	35
Bảng 4.2	Thống kê mã nguồn của hệ thống Smart Grading	36
Bảng 4.3	Tổng hợp kết quả kiểm thử các chức năng chính	37
Bảng A.1	Table to test captions and labels.	58

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
AMC	Auto-Multiple-Choice - gói LaTeX hỗ trợ tạo đề và phiếu trả lời trắc nghiệm
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CDN	Mạng phân phối nội dung (Content Delivery Network)
CRUD	Create - Read - Update - Delete: bốn thao tác cơ bản trên dữ liệu
CSS	Cascading Style Sheets - ngôn ngữ định dạng giao diện web
CV	Thị giác máy tính (Computer Vision)
DOM	Document Object Model - mô hình đối tượng tài liệu trong trình duyệt web
DPI	Dots Per Inch - số điểm ảnh trên mỗi inch, đơn vị đo độ phân giải in ấn
JWT	JSON Web Token - chuẩn mã thông báo truy cập dạng JSON
LaTeX	Hệ thống sắp chữ khoa học dựa trên TeX, dùng để soạn thảo tài liệu kỹ thuật
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
MVC	Model-View-Controller - mô hình kiến trúc phần mềm phân tách dữ liệu, giao diện và điều khiển
ODM	Object Document Mapper - lớp ánh xạ giữa đối tượng và tài liệu trong CSDL
OMR	Nhận dạng dấu quang học (Optical Mark Recognition)
OpenCV	Thư viện mã nguồn mở dùng cho thị giác máy tính và xử lý ảnh (Open Source Computer Vision Library)

Thuật ngữ	Ý nghĩa
ORM	Object Relational Mapper - lớp ánh xạ giữa đối tượng và quan hệ trong CSDL
REST	Representational State Transfer - kiểu kiến trúc API phổ biến trên HTTP
SDK	Bộ công cụ phát triển phần mềm (Software Development Kit)
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)
WSL2	Windows Subsystem for Linux phiên bản 2 - môi trường Linux chạy trên Windows

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan chương

Chương 1 mở đầu đề án bằng việc trình bày bối cảnh thực tiễn của bài toán chấm thi trắc nghiệm hiện nay tại các trường phổ thông và đại học ở Việt Nam, từ đó nêu bật tính cấp thiết và quy mô của vấn đề mà đề án hướng tới giải quyết. Trên cơ sở đó, chương này xác định rõ mục tiêu, phạm vi, định hướng giải pháp và bố cục tổng thể của đề án, nhằm giúp người đọc có cái nhìn tổng quan trước khi đi vào các nội dung chi tiết ở các chương tiếp theo.

1.2 Đặt vấn đề

Hình thức thi trắc nghiệm khách quan đang được sử dụng rộng rãi trong các kỳ thi tốt nghiệp, kỳ thi tuyển sinh đại học, các kỳ kiểm tra định kỳ tại các trường phổ thông và đại học ở Việt Nam nhờ ưu điểm khách quan, có thể chấm tự động và phù hợp với số lượng thí sinh lớn. Theo thống kê của Bộ Giáo dục và Đào tạo, mỗi năm hàng triệu thí sinh tham gia kỳ thi tốt nghiệp trung học phổ thông quốc gia, và mỗi trường đại học thường tổ chức hàng chục kỳ thi, kiểm tra với số lượng bài thi lên tới hàng trăm nghìn bài mỗi năm. Đi cùng với quy mô đó là nhu cầu tổ chức thi hiệu quả, minh bạch và giảm thiểu sai sót trong khâu chấm bài.

Tuy nhiên, quy trình chấm thi trắc nghiệm hiện nay tại nhiều trường vẫn còn tồn tại nhiều hạn chế đáng chú ý. Nhiều trường vẫn chấm bài thủ công hoặc bán thủ công trên phiếu điền tay, vừa tốn thời gian vừa dễ phát sinh sai sót do con người. Các giải pháp chấm thi bằng máy quét OMR chuyên dụng thường có chi phí đầu tư ban đầu lớn, đòi hỏi phần cứng riêng và khó triển khai đồng bộ cho các trường có quy mô nhỏ hoặc nguồn lực hạn chế. Ngoài ra, việc xử lý phúc khảo, thống kê kết quả và phân tích chất lượng đề thi cũng chưa được số hóa triệt để, dẫn đến thiếu thông tin phản hồi cho giáo viên trong việc cải tiến chất lượng giảng dạy và ra đề.

Những hạn chế nêu trên cho thấy nhu cầu cấp thiết về một hệ thống chấm thi trắc nghiệm thông minh, có khả năng tự động hóa toàn bộ quy trình từ khâu tạo đề thi đến chấm bài, xử lý phúc khảo và cung cấp báo cáo thống kê chi tiết. Hệ thống như vậy sẽ giúp giảm tải công việc cho giáo viên, hạn chế sai sót, tăng tính minh bạch cho học sinh, đồng thời cung cấp dữ liệu phong phú cho nhà trường trong việc đánh giá chất lượng đào tạo.

1.3 Mục tiêu và phạm vi đề tài

Hiện nay đã có một số phần mềm thương mại và mã nguồn mở hỗ trợ chấm thi trắc nghiệm trên thị trường, điển hình như các phần mềm chấm thi sử dụng máy

quét chuyên dụng (ví dụ: Remark Office OMR, ReadSoft, Gradescope) hoặc các công cụ mã nguồn mở xử lý OMR như OMRChecker. Bên cạnh đó, các hệ thống quản lý học tập (LMS) truyền thống như Moodle hay Google Forms cũng hỗ trợ tạo đề và chấm trắc nghiệm, nhưng thường giới hạn ở dạng thi trực tuyến thay vì xử lý phiếu trả lời giấy.

Qua khảo sát sơ bộ, có thể nhận thấy các giải pháp hiện có còn tồn tại các hạn chế chính như sau:

- phụ thuộc vào phần cứng máy quét chuyên dụng nên chi phí đầu tư cao và khó mở rộng;
- chưa hỗ trợ tốt việc tạo nhiều phiên bản đề thi từ ngân hàng câu hỏi để hạn chế gian lận;
- quy trình phức tạp chưa được số hóa và thiếu tính minh bạch;
- thiếu tích hợp giữa phần quản lý đào tạo (lớp, học sinh, giáo viên) với phần chấm thi;
- hầu hết các giải pháp chỉ chạy trên nền tảng web, chưa có ứng dụng di động cho phép giáo viên chấm bài ngay tại lớp học bằng chính chiếc điện thoại của mình.

Từ những hạn chế này, mục tiêu chính của đề án là xây dựng hệ thống chấm thi trắc nghiệm thông minh có tên gọi Smart Grading, với các chức năng chính gồm:

- quản lý trường học, lớp học, giáo viên và học sinh;
- quản lý ngân hàng câu hỏi và đề thi có hỗ trợ tạo câu hỏi bằng trí tuệ nhân tạo;
- tự động tạo nhiều phiên bản đề thi khác nhau từ cùng một ngân hàng câu hỏi;
- chấm thi tự động bằng nhận dạng OMR trên thiết bị di động;
- hỗ trợ quy trình tạo đơn phúc khảo và phản hồi phúc khảo minh bạch;
- tạo báo cáo thống kê chi tiết cho từng bài kiểm tra;
- cho phép học sinh xem bài thi và điểm số của mình.

Phạm vi đề tài tập trung vào môi trường thi trên giấy với phiếu trả lời dạng OMR, sử dụng tiếng Việt và phù hợp với bậc phổ thông trung học và đại học.

1.4 Định hướng giải pháp

Dựa trên mục tiêu và phạm vi đã xác định, đề án lựa chọn hướng tiếp cận xây dựng hệ thống đa nền tảng theo kiến trúc ba tầng, trong đó tầng giao diện gồm ứng dụng di động Flutter và giao diện web React, tầng xử lý nghiệp vụ là backend

Node.js/Express với cơ sở dữ liệu MongoDB, và tăng sinh đề thi được thực hiện bằng gói LaTeX AMC (Auto-Multiple-Choice) biên dịch qua WSL2. Định hướng công nghệ này được lựa chọn dựa trên ba lý do chính:

- cho phép tận dụng camera trên điện thoại thông minh để chấm OMR mà không cần đầu tư máy quét riêng;
- AMC hỗ trợ tạo nhiều phiên bản đề thi tự động, giải quyết bài toán chống gian lận;
- Node.js/Express và MongoDB giúp phát triển nhanh, dễ mở rộng và có hệ sinh thái thư viện phong phú.

Giải pháp thực tế mà đề án xây dựng là một hệ thống end-to-end, trong đó giáo viên sử dụng giao diện web để tạo đề thi từ ngân hàng câu hỏi, hệ thống tự động sinh nhiều mã đề và phiếu OMR tương ứng, giáo viên in phiếu và tổ chức thi, sau khi thu phiếu sử dụng ứng dụng di động để chụp và chấm bằng thuật toán nhận dạng OMR, kết quả được đồng bộ về server và cung cấp cho học sinh xem điểm, gửi phúc khảo nếu cần, đồng thời giúp giáo viên thống kê, phân tích chất lượng đề thi và đưa ra báo cáo chi tiết.

Đóng góp chính của đề án gồm:

- xây dựng thành công hệ thống tích hợp đầy đủ chu trình tạo đề, chấm thi, phúc khảo và báo cáo trên cùng một nền tảng;
- đề xuất và triển khai phương pháp sinh tự động phiếu OMR kèm tọa độ nhận dạng dựa trên AMC, giúp rút ngắn đáng kể thời gian chuẩn bị thi;
- triển khai thuật toán chấm OMR trên thiết bị di động có độ chính xác cao và khả năng phát hiện lỗi tô nhầm, tô nhẹ;
- tích hợp trợ lý AI hỗ trợ sinh câu hỏi và phân tích bài làm.

Kết quả dự kiến là một hệ thống hoàn chỉnh có thể triển khai thử nghiệm trong môi trường học thuật thực tế.

1.5 Bố cục đề án

Phần còn lại của đề án được tổ chức thành các chương tiếp theo. Chương 2 trình bày kết quả khảo sát hiện trạng, tổng quan các chức năng của hệ thống thông qua biểu đồ use case, biểu đồ phân rã cho từng use case chính cùng đặc tả chi tiết cho các use case quan trọng nhất; ngoài ra, chương này cũng nêu các yêu cầu phi chức năng làm cơ sở cho thiết kế và triển khai.

Trong Chương 3, đề án giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng, bao gồm: kỹ thuật nhận dạng OMR và xử lý ảnh, nguyên lý tạo đề nhiều

phiên bản với AMC, các công nghệ chính trong backend Node.js/Express, frontend React, ứng dụng di động Flutter, cùng với phân tích lý do lựa chọn từng công nghệ và so sánh với các lựa chọn thay thế.

Chương 4 trình bày chi tiết quá trình phân tích thiết kế kiến trúc phần mềm, thiết kế giao diện, thiết kế lớp với biểu đồ trình tự cho các use case chính, thiết kế cơ sở dữ liệu, đồng thời mô tả kết quả xây dựng ứng dụng, minh họa các chức năng chính, kết quả kiểm thử và triển khai thử nghiệm trong môi trường thực tế.

Chương 5 tập trung trình bày các giải pháp và đóng góp nổi bật của đề án, trong đó phân tích chi tiết các bài toán khó đã giải quyết như: thuật toán trộn đề nhiều phiên bản với đảm bảo độ khó đồng đều, quy trình tự động sinh tọa độ phiếu OMR, thuật toán nhận dạng OMR trên thiết bị di động, và mô hình tích hợp trợ lý AI vào quy trình tạo đề và phản hồi phúc khảo.

Cuối cùng, Chương 6 tổng kết những kết quả đạt được của đề án, đối chiếu với các sản phẩm tương tự, rút ra bài học kinh nghiệm trong quá trình thực hiện, đồng thời đề xuất hướng phát triển trong tương lai để hoàn thiện hệ thống và mở rộng phạm vi ứng dụng.

1.6 Kết chương

Chương 1 đã trình bày tổng quan bối cảnh thực tiễn của bài toán chấm thi trắc nghiệm tại Việt Nam, qua đó làm nổi bật tính cấp thiết của việc xây dựng một hệ thống tích hợp đầy đủ từ quản lý đào tạo đến chấm thi và phúc khảo. Trên cơ sở đánh giá các giải pháp hiện có, chương này đã xác định rõ mục tiêu, phạm vi của hệ thống Smart Grading, đồng thời nêu định hướng giải pháp đa nền tảng kết hợp công nghệ nhận dạng OMR trên di động và tích hợp AI. Các kết quả chính được tổng kết gồm:

- hệ thống hóa bài toán và mục tiêu rõ ràng;
- xác định bảy nhóm chức năng chính cần xây dựng;
- định hướng công nghệ khả thi cho từng thành phần.

Chương 2 sẽ trình bày chi tiết kết quả khảo sát hiện trạng và phân tích yêu cầu của hệ thống, làm cơ sở cho các quyết định thiết kế ở các chương sau.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Tổng quan chương

Chương 1 đã giới thiệu bối cảnh thực tiễn, hạn chế của các giải pháp hiện có và định hướng giải pháp của hệ thống Smart Grading. Dựa trên những phân tích đó, Chương 2 trình bày chi tiết kết quả khảo sát hiện trạng và phân tích yêu cầu của hệ thống. Nội dung chương gồm: tổng quan các sản phẩm tương tự và nhu cầu thực tế của người dùng; mô tả tổng quan chức năng của hệ thống thông qua biểu đồ use case tổng quát, biểu đồ phân rã cho từng nhóm chức năng chính và quy trình nghiệp vụ trọng yếu; đặc tả chi tiết cho các use case quan trọng; và cuối cùng là các yêu cầu phi chức năng làm cơ sở cho việc thiết kế và triển khai ở các chương sau.

2.2 Khảo sát hiện trạng

Để xác định rõ yêu cầu cho hệ thống Smart Grading, đề án tiến hành khảo sát hiện trạng từ ba nguồn thông tin chính, bao gồm:

- khảo sát nhu cầu của giáo viên và học sinh tại một số trường phổ thông và đại học;
- khảo sát hệ thống chấm thi đang được sử dụng tại trường;
- khảo sát các ứng dụng tương tự trên thị trường.

Khảo sát nhu cầu người dùng. Qua khảo sát trực tuyến và phỏng vấn sơ bộ khoảng 30 giáo viên và 80 học sinh, có thể nhận thấy các nhu cầu phổ biến như sau: giáo viên mong muốn có công cụ tạo đề thi nhanh từ ngân hàng câu hỏi có sẵn, hỗ trợ sinh nhiều mã đề để hạn chế gian lận, có khả năng chấm bài ngay sau khi kết thúc buổi thi để công bố kết quả sớm; học sinh mong muốn được xem điểm và bài làm của mình một cách minh bạch, có cơ chế gửi phúc khảo rõ ràng khi cho rằng điểm chưa chính xác. Ngoài ra, cả giáo viên và học sinh đều mong muốn hệ thống hỗ trợ thống kê kết quả theo lớp, theo môn, theo chủ đề để phục vụ cho việc đánh giá chất lượng học tập.

Khảo sát hệ thống hiện tại. Tại các trường được khảo sát, hệ thống chấm thi hiện đang sử dụng chủ yếu thuộc hai loại:

- chấm thủ công bằng cách đối chiếu phiếu trả lời với đáp án, áp dụng phổ biến tại các trường có quy mô nhỏ;
- chấm bằng máy quét OMR chuyên dụng kết hợp phần mềm đi kèm, áp dụng tại một số trường lớn và các hội đồng thi cấp tỉnh.

Hệ thống loại thứ nhất có ưu điểm chi phí thấp nhưng tốn thời gian, dễ sai sót và khó thống kê. Hệ thống loại thứ hai cho tốc độ chấm nhanh và độ chính xác cao, nhưng chi phí đầu tư máy quét lớn, phần mềm thường đi kèm không hỗ trợ tốt các nghiệp vụ quản lý đào tạo và phúc khảo.

Khảo sát ứng dụng tương tự. Đề án khảo sát một số giải pháp chấm thi phổ biến hiện nay, gồm: phần mềm thương mại như Remark Office OMR và Gradescope; công cụ mã nguồn mở như OMRChecker; các hệ thống thi trực tuyến như Google Forms, Microsoft Forms và Quizizz. Đặc điểm so sánh giữa các giải pháp được tổng hợp trong Bảng 2.1.

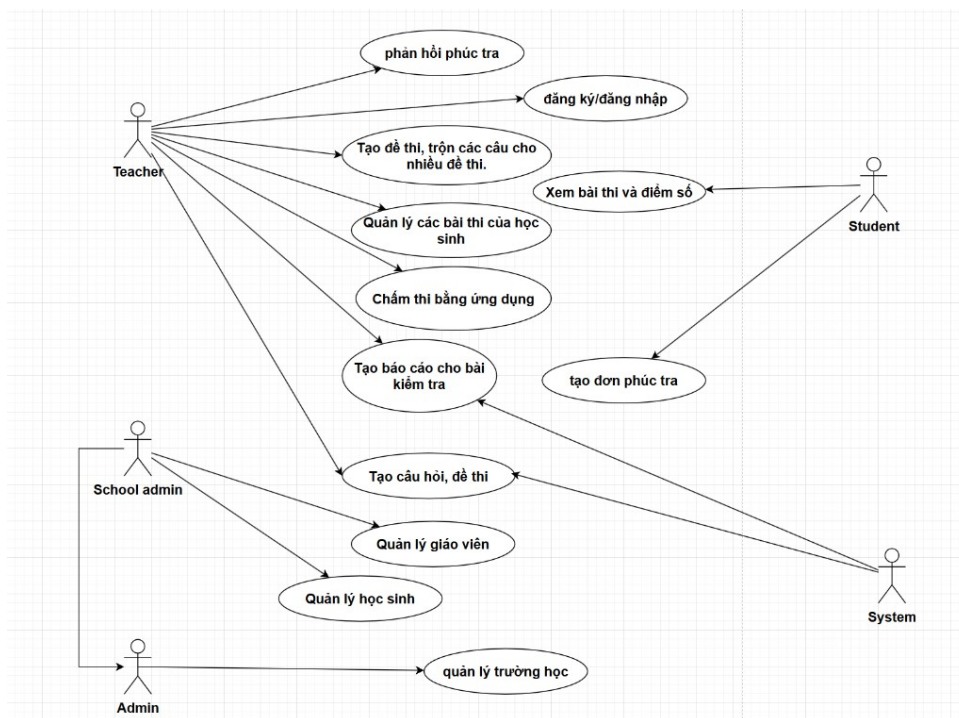
Tính năng	Remark OMR	Gradescope	OMRChecker	Smart Grading
Chấm OMR tự động	Có	Có	Có	Có
Tạo nhiều mã đề	Không	Không	Không	Có
Quản lý lớp, học sinh	Không	Có	Không	Có
Phúc khảo trực tuyến	Không	Có	Không	Có
Chấm trên di động	Không	Có	Có	Có
Hỗ trợ tiếng Việt	Không	Không	Một phần	Có
Tích hợp AI	Không	Có	Không	Có
Chi phí đầu tư	Cao	Cao	Miễn phí	Miễn phí

Bảng 2.1: So sánh tính năng giữa các giải pháp chấm thi hiện có

Qua bảng so sánh có thể nhận thấy hệ thống Smart Grading được đề xuất trong đề án có sự kết hợp đầy đủ các tính năng mà các giải pháp hiện có chưa đáp ứng đồng thời, đặc biệt là khả năng tạo nhiều mã đề, quản lý đào tạo tích hợp, hỗ trợ phúc khảo minh bạch, chấm trên di động và hỗ trợ tiếng Việt đầy đủ.

2.3 Tổng quan chức năng

Trên cơ sở kết quả khảo sát, hệ thống Smart Grading được đề xuất với các nhóm chức năng chính gồm: quản lý người dùng và đào tạo (trường, lớp, giáo viên, học sinh), quản lý ngân hàng câu hỏi và đề thi, chấm thi bằng nhận dạng OMR, quản lý bài thi và phúc khảo, và báo cáo thống kê. Biểu đồ use case tổng quát mô tả các tác nhân và use case chính của hệ thống như trong Hình 2.1.



Hình 2.1: Biểu đồ use case tổng quát của hệ thống Smart Grading

Tác nhân tham gia hệ thống. Hệ thống có bốn tác nhân chính:

- *Quản trị viên hệ thống* (admin): chịu trách nhiệm quản lý toàn bộ trường, tài khoản và cấu hình hệ thống;
- *Quản trị viên trường* (school-admin): quản lý các lớp, giáo viên, học sinh thuộc trường mình phụ trách;
- *Giáo viên* (teacher): thực hiện tạo đề thi, quản lý ngân hàng câu hỏi, chấm bài và xử lý phúc khảo;
- *Học sinh* (student): xem đề thi, xem điểm, gửi yêu cầu phúc khảo và sử dụng trợ lý AI.

Các use case chính. Hệ thống có 13 use case chính, được phân thành 4 nhóm chức năng lớn và sẽ được phân rõ chi tiết ở các mục tiếp theo:

- *Nhóm quản lý đào tạo:* Quản lý trường học, Quản lý giáo viên, Quản lý học sinh, Đăng ký/Đăng nhập;
- *Nhóm ngân hàng câu hỏi và đề thi:* Tạo câu hỏi/đề thi (có tương tác System), Tạo đề thi và trộn câu cho nhiều đề thi;
- *Nhóm chấm thi và phúc khảo:* Quản lý các bài thi của học sinh, Chấm thi bằng ứng dụng, Tạo đơn phúc khảo, Phản hồi phúc khảo, Xem bài thi và điểm số;
- *Nhóm thống kê:* Tạo báo cáo cho bài kiểm tra (có tương tác System).

2.3.1 Biểu đồ use case phân rã Đăng ký/Đăng nhập

Use case Đăng ký/Đăng nhập bao gồm các use case con:

- Đăng ký tài khoản;
- Đăng nhập hệ thống;
- Xác thực email;
- Đặt lại mật khẩu;
- Đăng xuất;
- Quản lý phiên truy cập.

Tác nhân chính là tất cả bốn vai trò người dùng.

2.3.2 Biểu đồ use case phân rã Quản lý trường học

Use case Quản lý trường học gồm:

- Tạo trường mới;
- Cập nhật thông tin trường;
- Xóa trường;
- Xem danh sách trường;
- Cấu hình thang điểm và tham số chấm.

Tác nhân chính là Quản trị viên hệ thống.

2.3.3 Biểu đồ use case phân rã Quản lý giáo viên

Use case Quản lý giáo viên gồm:

- Thêm giáo viên;
- Cập nhật thông tin giáo viên;
- Phân công giáo viên theo môn;
- Vô hiệu hóa tài khoản giáo viên;
- Xem danh sách giáo viên.

Tác nhân chính là Quản trị viên hệ thống và Quản trị viên trường.

2.3.4 Biểu đồ use case phân rã Quản lý học sinh

Use case Quản lý học sinh gồm:

- Thêm học sinh vào lớp;
- Cập nhật thông tin học sinh;

- Chuyển lớp cho học sinh;
- Xóa học sinh khỏi lớp;
- Xem danh sách học sinh theo lớp.

Tác nhân chính là Quản trị viên trường và Giáo viên.

2.3.5 Biểu đồ use case phân rã Tạo câu hỏi, đề thi

Use case Tạo câu hỏi và đề thi có sự tương tác của System, gồm:

- Tạo câu hỏi thủ công;
- Sinh câu hỏi bằng AI;
- Phê duyệt câu hỏi;
- Nhập khẩu câu hỏi từ file;
- Tạo bộ đề thi;
- Cấu hình tham số đề thi.

Tác nhân chính là Giáo viên, có sự hỗ trợ của System thông qua các dịch vụ AI.

2.3.6 Biểu đồ use case phân rã Tạo đề thi, trộn các câu cho nhiều đề thi

Use case Tạo đề thi và trộn câu cho nhiều đề thi gồm:

- Chọn câu hỏi từ ngân hàng;
- Cấu hình số lượng phiên bản;
- Thiết lập cấu hình trộn (câu hỏi, đáp án);
- Sinh phiếu OMR cho mỗi phiên bản;
- Phát hành đề thi;
- Xuất bản đề thi sang LaTeX/PDF.

Tác nhân chính là Giáo viên.

2.3.7 Biểu đồ use case phân rã Quản lý các bài thi của học sinh

Use case Quản lý các bài thi của học sinh gồm:

- Ghi nhận bài nộp từ thiết bị di động;
- Xem danh sách bài nộp;
- Chỉnh sửa thủ công kết quả chấm;
- Phê duyệt kết quả bài làm;
- Xóa bài nộp.

Tác nhân chính là Giáo viên.

2.3.8 Biểu đồ use case phân rã Chấm thi bằng ứng dụng

Use case Chấm thi bằng ứng dụng gồm:

- Chụp ảnh phiếu trả lời bằng camera;
- Tiền xử lý ảnh (làm phẳng, cắt, nhị phân hóa);
- Nhận dạng mã đề và mã học sinh;
- Nhận dạng các lựa chọn được tô;
- Đối chiếu đáp án và tính điểm;
- Đồng bộ kết quả lên server.

Tác nhân chính là Giáo viên thông qua thiết bị di động.

2.3.9 Biểu đồ use case phân rã Tạo đơn phúc khảo

Use case Tạo đơn phúc khảo gồm:

- Chọn bài thi cần phúc khảo;
- Chọn câu hỏi cần phúc khảo;
- Nhập lý do phúc khảo;
- Đính kèm hình ảnh bằng chứng (nếu có);
- Gửi đơn phúc khảo;
- Theo dõi trạng thái đơn.

Tác nhân chính là Học sinh.

2.3.10 Biểu đồ use case phân rã Phản hồi phúc khảo

Use case Phản hồi phúc khảo gồm:

- Tiếp nhận đơn phúc khảo;
- Xem chi tiết bài làm của học sinh;
- So sánh với đáp án;
- Phê duyệt hoặc từ chối;
- Nhập phản hồi cho học sinh;
- Cập nhật lại điểm (nếu phê duyệt).

Tác nhân chính là Giáo viên.

2.3.11 Biểu đồ use case phân rã Xem bài thi và điểm số

Use case Xem bài thi và điểm số gồm:

- Xem danh sách bài thi;
- Xem điểm tổng và điểm từng câu;
- Xem ảnh phiếu đã chấm;
- Xem đáp án đúng;
- Tải về báo cáo điểm cá nhân.

Tác nhân chính là Học sinh.

2.3.12 Biểu đồ use case phân rã Tạo báo cáo cho bài kiểm tra

Use case Tạo báo cáo cho bài kiểm tra có sự tương tác của System, gồm:

- Chọn bài kiểm tra cần báo cáo;
- Yêu cầu hệ thống tổng hợp dữ liệu;
- System phân tích thống kê (phổ điểm, độ khó câu hỏi, độ phân biệt);
- Sinh báo cáo dạng biểu đồ và bảng số liệu;
- Xuất báo cáo ra PDF/Excel.

Tác nhân chính là Giáo viên, có sự hỗ trợ của System thông qua các dịch vụ phân tích.

2.3.13 Quy trình nghiệp vụ

Hệ thống Smart Grading có một số quy trình nghiệp vụ quan trọng kết hợp nhiều use case với nhau. Quy trình nghiệp vụ "Tổ chức kỳ thi và chấm bài" được mô tả tổng quan như sau: đầu tiên giáo viên tạo đề thi từ ngân hàng câu hỏi, hệ thống tự động trộn đề thành nhiều phiên bản và sinh phiếu OMR tương ứng; giáo viên in phiếu và phát cho học sinh làm bài; sau khi thu phiếu, giáo viên sử dụng ứng dụng di động để chấm từng phiếu; kết quả chấm được đồng bộ về server và công bố cho học sinh; học sinh có thể xem điểm và gửi đơn phúc khảo nếu cần; giáo viên tiếp nhận và phản hồi phúc khảo; cuối cùng giáo viên tạo báo cáo thống kê cho bài kiểm tra.

Quy trình nghiệp vụ "Quản lý đào tạo" gồm: admin tạo trường học và tài khoản quản trị trường; quản trị trường tạo năm học, khối lớp, môn học và tài khoản giáo viên; giáo viên được phân công phụ trách lớp và môn học, tạo lớp và thêm học sinh vào lớp; học sinh được cấp tài khoản và đăng nhập vào hệ thống để xem thông tin cá nhân, kết quả học tập.

Quy trình nghiệp vụ "Phúc khảo" gồm: học sinh xem bài thi và phát hiện điểm chưa chính xác, gửi đơn phúc khảo kèm lý do và bằng chứng; giáo viên nhận thông báo, mở chi tiết bài làm của học sinh, so sánh với đáp án; giáo viên phê duyệt hoặc từ chối đơn phúc khảo kèm phản hồi; nếu phê duyệt, điểm của học sinh được cập nhật lại và thông báo kết quả được gửi cho học sinh.

2.4 Đặc tả chức năng

Căn cứ vào tầm quan trọng và độ phức tạp, đề án lựa chọn đặc tả chi tiết cho 6 use case chính gồm:

- Đăng ký/Đăng nhập;
- Tạo đề thi, trộn câu cho nhiều đề thi;
- Chấm thi bằng ứng dụng;
- Tạo đơn phúc khảo;
- Phản hồi phúc khảo;
- Tạo báo cáo cho bài kiểm tra.

Mỗi đặc tả gồm tên use case, luồng sự kiện chính, luồng sự kiện phát sinh, tiền điều kiện và hậu điều kiện.

2.4.1 Đặc tả use case Đăng ký/Đăng nhập

Tên use case: Đăng ký/Đăng nhập.

Tiền điều kiện: Người dùng truy cập được vào ứng dụng web hoặc ứng dụng di động Smart Grading.

Luồng sự kiện chính:

1. Người dùng truy cập trang đăng nhập và chọn chức năng "Đăng ký" nếu chưa có tài khoản.
2. Hệ thống hiển thị form đăng ký yêu cầu nhập họ tên, email, mật khẩu và xác nhận mật khẩu.
3. Người dùng nhập thông tin và gửi yêu cầu đăng ký.
4. Hệ thống kiểm tra tính hợp lệ của email, độ mạnh của mật khẩu và sự trùng lặp email trong cơ sở dữ liệu.
5. Hệ thống tạo tài khoản mới với trạng thái chưa xác thực email, gửi email xác thực chứa mã token đến email của người dùng.
6. Người dùng truy cập email, nhấp vào liên kết xác thực.
7. Hệ thống đánh dấu tài khoản đã xác thực email và hiển thị thông báo xác thực

thành công.

8. Người dùng quay lại trang đăng nhập, nhập email và mật khẩu để đăng nhập.
9. Hệ thống kiểm tra thông tin đăng nhập, cấp access token và refresh token cho người dùng.
10. Hệ thống chuyển hướng người dùng đến trang dashboard tương ứng với vai trò của họ.

Luồng sự kiện phát sinh:

- Tại bước 4, nếu email đã tồn tại: hệ thống thông báo email đã được đăng ký và đề xuất chức năng đăng nhập hoặc đặt lại mật khẩu.
- Tại bước 4, nếu mật khẩu không đủ mạnh: hệ thống yêu cầu người dùng nhập lại mật khẩu theo yêu cầu bảo mật.
- Tại bước 7, nếu liên kết xác thực đã hết hạn: hệ thống cho phép gửi lại email xác thực mới.
- Tại bước 9, nếu thông tin đăng nhập sai: hệ thống thông báo sai thông tin và cho phép thử lại tối đa 5 lần trước khi khóa tài khoản tạm thời.

Hậu điều kiện: Nếu thành công, người dùng được đăng nhập vào hệ thống và có quyền truy cập các chức năng tương ứng với vai trò. Nếu thất bại, trạng thái hệ thống không thay đổi.

2.4.2 Đặc tả use case Tạo đề thi, trộn các câu cho nhiều đề thi

Tên use case: Tạo đề thi, trộn các câu cho nhiều đề thi.

Tiền điều kiện: Giáo viên đã đăng nhập vào hệ thống, đã có ngân hàng câu hỏi thuộc môn học tương ứng.

Luồng sự kiện chính:

1. Giáo viên chọn chức năng "Tạo đề thi" trên giao diện web.
2. Hệ thống hiển thị form tạo đề thi với các trường: tiêu đề, mô tả, lớp áp dụng, môn học, thời gian làm bài, tổng điểm, số lượng phiên bản cần tạo.
3. Giáo viên nhập thông tin chung và lưu bản nháp.
4. Giáo viên chọn câu hỏi từ ngân hàng câu hỏi theo chủ đề, độ khó hoặc theo danh sách đã chuẩn bị trước.
5. Giáo viên cấu hình tham số trộn đề: có trộn thứ tự câu hỏi hay không, có trộn thứ tự đáp án hay không, số lượng phiên bản cần tạo (tối đa 50).
6. Giáo viên nhấn nút "Sinh đề thi" để yêu cầu hệ thống tạo các phiên bản đề.

7. Hệ thống tạo các phiên bản đề thi bằng cách hoán vị câu hỏi và đáp án theo cấu hình, đảm bảo đáp án đúng được cập nhật tương ứng cho từng phiên bản.
8. Hệ thống sinh mã nguồn LaTeX sử dụng gói AMC cho từng phiên bản đề, biên dịch sang PDF và tải lên dịch vụ lưu trữ đám mây.
9. Hệ thống sinh phiếu trả lời OMR kèm tọa độ nhận dạng cho từng phiên bản, lưu thông tin tọa độ vào cơ sở dữ liệu.
10. Giáo viên xem lại danh sách các phiên bản đề đã tạo, tải về file PDF đề thi và phiếu trả lời.
11. Giáo viên nhấn nút "Phát hành" để công bố đề thi cho học sinh.
12. Hệ thống gửi thông báo đến học sinh thuộc các lớp áp dụng và cập nhật trạng thái đề thi thành "đã phát hành".

Luồng sự kiện phát sinh:

- Tại bước 3, nếu thiếu thông tin bắt buộc: hệ thống yêu cầu giáo viên bổ sung.
- Tại bước 4, nếu ngân hàng câu hỏi không đủ số lượng cần thiết: hệ thống cảnh báo và đề xuất giảm số lượng phiên bản.
- Tại bước 7, nếu có lỗi khi trộn đề: hệ thống thông báo và giữ nguyên bản nháp.
- Tại bước 8, nếu biên dịch LaTeX thất bại: hệ thống ghi nhận log lỗi và cho phép giáo viên thử lại.

Hậu điều kiện: Nếu thành công, đề thi được tạo với nhiều phiên bản, phiếu OMR tương ứng được sinh và học sinh nhận được thông báo. Nếu thất bại, trạng thái đề thi giữ nguyên ở bản nháp.

2.4.3 Đặc tả use case Chấm thi bằng ứng dụng

Tên use case: Chấm thi bằng ứng dụng.

Tiền điều kiện: Giáo viên đã đăng nhập trên ứng dụng di động, có bài thi đã được phát hành và phiếu OMR tương ứng.

Luồng sự kiện chính:

1. Giáo viên mở ứng dụng di động Smart Grading và chọn chức năng "Quét phiếu".
2. Ứng dụng yêu cầu quyền truy cập camera và hiển thị giao diện chụp ảnh.
3. Giáo viên đặt phiếu trả lời trên bề mặt phẳng, căn chỉnh vào khung hình và chụp ảnh.
4. Ứng dụng tải thông tin mẫu OMR tương ứng với bài thi từ server.

5. Ứng dụng thực hiện tiền xử lý ảnh: chuyển ảnh sang thang xám, làm mờ Gaussian, nhị phân hóa bằng phương pháp Otsu, phát hiện và chỉnh nghiêng phiếu, cắt vùng phiếu.
6. Ứng dụng phát hiện các ô trả lời bằng cách so sánh vị trí với tọa độ đã lưu trong mẫu OMR.
7. Ứng dụng đánh giá mức độ tô của từng ô, xác định ô được chọn dựa trên ngưỡng diện tích tô, phát hiện lỗi tô nhầm (nhiều ô trong cùng một câu) hoặc tô nhẹ.
8. Ứng dụng nhận dạng mã đề và mã số sinh viên từ vùng đầu phiếu.
9. Ứng dụng đối chiếu câu trả lời của học sinh với đáp án của phiên bản đề tương ứng, tính điểm cho từng câu và tổng điểm.
10. Ứng dụng hiển thị kết quả chấm cho giáo viên kèm ảnh phiếu đã được đánh dấu.
11. Giáo viên kiểm tra và có thể chỉnh sửa thủ công các câu bị nhận dạng sai hoặc đánh dấu là không hợp lệ.
12. Giáo viên nhấn nút "Lưu kết quả" để đồng bộ kết quả lên server.

Luồng sự kiện phát sinh:

- Tại bước 5, nếu ảnh chụp mờ hoặc nghiêng quá nhiều: ứng dụng yêu cầu giáo viên chụp lại.
- Tại bước 8, nếu không nhận dạng được mã đề: ứng dụng cho phép giáo viên chọn thủ công mã đề.
- Tại bước 8, nếu phát hiện câu tô nhầm hoặc tô nhẹ: ứng dụng đánh dấu cảnh báo và yêu cầu giáo viên xác nhận.
- Tại bước 13, nếu mất kết nối mạng: ứng dụng lưu kết quả cục bộ và tự động đồng bộ khi có kết nối.

Hậu điều kiện: Nếu thành công, bài làm của học sinh được lưu trên server với điểm số đã tính, học sinh có thể xem điểm sau khi giáo viên phát hành kết quả. Nếu thất bại, ứng dụng giữ kết quả ở trạng thái chờ xử lý.

2.4.4 Đặc tả use case Tạo đơn phúc khảo

Tên use case: Tạo đơn phúc khảo.

Tiền điều kiện: Học sinh đã đăng nhập vào hệ thống, đã có kết quả bài thi được công bố và còn trong thời hạn phúc khảo.

Luồng sự kiện chính:

1. Học sinh truy cập chức năng "Bài thi của tôi" và chọn bài thi muốn phúc khảo.
2. Hệ thống hiển thị chi tiết bài thi gồm: điểm tổng, điểm từng câu, ảnh phiếu đã chấm và đáp án đúng.
3. Học sinh chọn chức năng "Tạo đơn phúc khảo" và chọn một hoặc nhiều câu cần phúc khảo.
4. Hệ thống hiển thị form yêu cầu nhập lý do phúc khảo cho từng câu, cho phép đính kèm hình ảnh minh chứng (nếu có).
5. Học sinh nhập lý do, đính kèm bằng chứng (nếu có) và nhấn nút "Gửi đơn".
6. Hệ thống kiểm tra thời hạn phúc khảo và tính hợp lệ của thông tin.
7. Hệ thống tạo đơn phúc khảo mới với trạng thái "đang chờ", gửi thông báo cho giáo viên phụ trách.
8. Hệ thống gửi thông báo xác nhận cho học sinh và hiển thị trạng thái đơn phúc khảo trong danh sách "Đơn phúc khảo của tôi".

Luồng sự kiện phát sinh:

- Tại bước 6, nếu hết thời hạn phúc khảo: hệ thống từ chối và thông báo thời hạn đã kết thúc.
- Tại bước 6, nếu học sinh đã gửi đơn cho cùng một câu: hệ thống từ chối và đề xuất xem trạng thái đơn cũ.
- Tại bước 4, nếu lý do phúc khảo quá ngắn: hệ thống yêu cầu nhập chi tiết hơn.

Hậu điều kiện: Nếu thành công, đơn phúc khảo được tạo và ở trạng thái "đang chờ", giáo viên nhận được thông báo để xử lý. Nếu thất bại, trạng thái các đơn cũ không thay đổi.

2.4.5 Đặc tả use case Phản hồi phúc khảo

Tên use case: Phản hồi phúc khảo.

Tiền điều kiện: Giáo viên đã đăng nhập, có đơn phúc khảo đang ở trạng thái "đang chờ".

Luồng sự kiện chính:

1. Giáo viên truy cập chức năng "Phúc khảo" và xem danh sách đơn phúc khảo đang chờ.
2. Giáo viên chọn một đơn phúc khảo để xem chi tiết.

3. Hệ thống hiển thị thông tin bài thi của học sinh, câu được phúc khảo, lý do và hình ảnh minh chứng (nếu có).
4. Giáo viên so sánh câu trả lời của học sinh với đáp án, đối chiếu với hình ảnh phiếu gốc.
5. Giáo viên quyết định phê duyệt hoặc từ chối và nhập nội dung phản hồi cho học sinh.
6. Giáo viên nhấn nút "Gửi phản hồi" để hoàn tất.
7. Hệ thống cập nhật trạng thái đơn phúc khảo thành "đã phê duyệt" hoặc "đã từ chối".
8. Nếu phê duyệt, hệ thống tính lại điểm cho câu được phúc khảo và cập nhật tổng điểm của học sinh.
9. Hệ thống gửi thông báo kết quả phúc khảo cho học sinh.

Luồng sự kiện phát sinh:

- Tại bước 5, nếu giáo viên cần thêm thời gian: đơn được đánh dấu "đang xử lý" và có thể tiếp tục sau.
- Tại bước 8, nếu có lỗi khi tính lại điểm: hệ thống ghi nhận lỗi, giữ nguyên điểm cũ và thông báo cho quản trị hệ thống.
- Tại bước 5, nếu đơn đã hết thời hạn xử lý: hệ thống cảnh báo và yêu cầu giáo viên xác nhận trước khi gửi.

Hậu điều kiện: Nếu thành công, đơn phúc khảo được đóng và điểm được cập nhật (nếu phê duyệt). Học sinh nhận được thông báo kết quả.

2.4.6 Đặc tả use case Tạo báo cáo cho bài kiểm tra

Tên use case: Tạo báo cáo cho bài kiểm tra.

Tiền điều kiện: Giáo viên đã đăng nhập, bài kiểm tra đã có kết quả chấm từ học sinh.

Luồng sự kiện chính:

1. Giáo viên truy cập chức năng "Báo cáo" và chọn bài kiểm tra cần tạo báo cáo.
2. Hệ thống tổng hợp dữ liệu: danh sách bài làm, điểm số, đáp án, thời gian làm bài.
3. Hệ thống (System) phân tích thống kê: tính phổ điểm, điểm trung bình, độ lệch chuẩn, tỉ lệ đạt, độ khó và độ phân biệt của từng câu hỏi.
4. Hệ thống sinh báo cáo dưới dạng biểu đồ (cột, tròn, phân tán) và bảng số liệu.

5. Hệ thống sử dụng AI để sinh nhận xét tổng quan về chất lượng đề thi và gợi ý cải thiện.
6. Giáo viên xem trước báo cáo, có thể tùy chỉnh các chỉ số muốn hiển thị.
7. Giáo viên chọn xuất báo cáo ra định dạng PDF hoặc Excel.
8. Hệ thống sinh file báo cáo và cho phép giáo viên tải về.

Luồng sự kiện phát sinh:

- Tại bước 3, nếu dữ liệu chưa đầy đủ: hệ thống cảnh báo và chỉ sinh báo cáo với các chỉ số khả dụng.
- Tại bước 5, nếu dịch vụ AI gặp sự cố: hệ thống bỏ qua phần nhận xét AI và tiếp tục sinh phần còn lại của báo cáo.
- Tại bước 7, nếu xuất file thất bại: hệ thống thông báo lỗi và cho phép thử lại.

Hậu điều kiện: Nếu thành công, giáo viên nhận được file báo cáo hoàn chỉnh để sử dụng cho công tác đánh giá chất lượng giảng dạy và ra đề.

2.5 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng đã nêu, hệ thống Smart Grading cần đáp ứng các yêu cầu phi chức năng sau:

Yêu cầu về hiệu năng. Hệ thống phải đảm bảo thời gian phản hồi của các API dưới 2 giây đối với các thao tác thông thường và dưới 5 giây đối với các thao tác tổng hợp dữ liệu lớn như tạo báo cáo. Thời gian chấm một phiếu trả lời bằng ứng dụng di động không quá 3 giây với điều kiện ảnh đầu vào đạt chất lượng.

Yêu cầu về độ tin cậy. Hệ thống phải đảm bảo độ chính xác của thuật toán nhận dạng OMR đạt tối thiểu 95% trên các điều kiện ảnh chụp thông thường. Hệ thống phải có cơ chế ghi log đầy đủ để truy vết lỗi và phải có khả năng khôi phục sau sự cố mà không mất dữ liệu đã chấm.

Yêu cầu về tính dễ dùng. Giao diện người dùng phải được Việt hóa hoàn toàn, sử dụng phông chữ dễ đọc và có độ tương phản phù hợp. Các thao tác chính phải được thực hiện trong vòng không quá 3 lần nhấp chuột hoặc chạm. Ứng dụng di động phải hoạt động ổn định trên các thiết bị có cấu hình trung bình với hệ điều hành Android từ 7.0 trở lên.

Yêu cầu về tính bảo trì. Mã nguồn phải được tổ chức theo kiến trúc phân lớp rõ ràng, có tài liệu hướng dẫn cài đặt và vận hành. Hệ thống phải có bộ kiểm thử tự động cho các thành phần cốt lõi với độ phủ kiểm thử tối thiểu 60%.

Yêu cầu về bảo mật. Hệ thống phải sử dụng JWT cho xác thực, mật khẩu phải

được lưu trữ dưới dạng mã băm bcrypt. Các điểm cuối API phải được phân quyền theo vai trò người dùng. Ảnh phiếu thi và dữ liệu cá nhân của học sinh phải được bảo vệ khỏi truy cập trái phép.

Yêu cầu về cơ sở dữ liệu và công nghệ. Hệ thống sử dụng cơ sở dữ liệu MongoDB phiên bản 5.0 trở lên để lưu trữ dữ liệu nghiệp vụ. Hệ thống phải hỗ trợ triển khai trên cả môi trường phát triển (local) và môi trường đám mây (cloud). Backend chạy trên Node.js v18 trở lên, frontend web xây dựng trên React 18 với TypeScript, ứng dụng di động xây dựng trên Flutter 3.x.

2.6 Kết chương

Chương 2 đã trình bày kết quả khảo sát hiện trạng từ ba nguồn: nhu cầu người dùng, hệ thống chấm thi hiện tại tại các trường và các ứng dụng tương tự trên thị trường. Trên cơ sở đó, chương này đã mô tả tổng quan chức năng của hệ thống Smart Grading với mười ba use case chính phân thành bốn nhóm, đồng thời đặc tả chi tiết sáu use case quan trọng nhất với đầy đủ luồng sự kiện chính, luồng phát sinh, tiền điều kiện và hậu điều kiện. Bên cạnh đó, các yêu cầu phi chức năng về hiệu năng, độ tin cậy, tính dễ dùng, bảo trì, bảo mật và công nghệ cũng đã được xác định rõ. Các kết quả chính của chương gồm:

- bảng so sánh tính năng giữa các giải pháp hiện có;
- biểu đồ use case tổng quát và mười hai biểu đồ phân rã cho từng nhóm use case;
- ba quy trình nghiệp vụ trọng yếu;
- sáu đặc tả use case chi tiết.

Chương 3 sẽ trình bày các nền tảng lý thuyết và công nghệ được lựa chọn để hiện thực hóa các yêu cầu đã nêu trong chương này.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

3.1 Tổng quan chương

Chương 2 đã trình bày chi tiết yêu cầu chức năng và phi chức năng của hệ thống Smart Graging. Trên cơ sở đó, Chương 3 giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng để hiện thực hóa các yêu cầu đã nêu. Với mỗi công nghệ, chương này phân tích rõ công nghệ đó được dùng để giải quyết vấn đề cụ thể nào đã nêu ở Chương 2, đồng thời so sánh với các lựa chọn thay thế để giải thích lý do lựa chọn. Toàn bộ nội dung chương được tham chiếu chéo với danh mục tài liệu tham khảo để tăng tính khoa học và độ tin cậy.

3.2 Kỹ thuật nhận dạng OMR và xử lý ảnh số

Kỹ thuật nhận dạng dấu quang học (Optical Mark Recognition - OMR) là nền tảng cốt lõi để giải quyết yêu cầu chấm thi tự động các phiếu trả lời trên giấy đã nêu trong Chương 2. Theo tài liệu của Adams [1], OMR là kỹ thuật sử dụng cảm biến quang học để phát hiện sự có mặt hoặc vắng mặt của dấu trên một vị trí xác định trước trên tờ giấy, từ đó suy ra lựa chọn của người trả lời. Kỹ thuật này đã được ứng dụng rộng rãi trong các bài kiểm tra trắc nghiệm, khảo sát ý kiến và bầu cử nhờ tốc độ xử lý nhanh và độ chính xác cao.

Quy trình xử lý OMR thường gồm các bước chính:

- thu nhận ảnh từ máy quét chuyên dụng hoặc camera;
- tiền xử lý ảnh để cải thiện chất lượng;
- phát hiện vùng phiếu và chỉnh nghiêng;
- phát hiện các ô trả lời;
- phân loại trạng thái tô của từng ô;
- ánh xạ câu trả lời.

Trong đồ án, các bước này được cài đặt trong module xử lý ảnh của ứng dụng di động Flutter và sử dụng thư viện OpenCV [2] thông qua package `opencv_dart` để thực hiện các phép biến đổi ảnh cơ bản. Tài liệu của Gonzalez và Woods [3] cung cấp nền tảng lý thuyết cho các phép biến đổi như chuyển ảnh xám, lọc Gaussian, nhị phân hóa Otsu và phát hiện cạnh Canny được sử dụng trong hệ thống.

So với các giải pháp OMR truyền thống phụ thuộc vào máy quét chuyên dụng, đồ án lựa chọn hướng tiếp cận sử dụng camera của thiết bị di động kết hợp với các thuật toán xử lý ảnh để thực hiện nhận dạng. Hướng tiếp cận này có ưu điểm là

giảm chi phí đầu tư phần cứng, dễ triển khai và cho phép giáo viên chấm bài ngay tại lớp học. Dự án mã nguồn mở OMRChecker [4] cung cấp tham khảo quý giá về pipeline xử lý OMR trên thiết bị di động, mặc dù tập trung vào định dạng phiếu cố định. Trong khi đó, hệ thống Smart Grading tự thiết kế pipeline linh hoạt để làm việc với nhiều dạng phiếu khác nhau được sinh tự động bởi gói AMC.

3.3 Kỹ thuật tạo đề nhiều phiên bản với AMC

Yêu cầu tạo nhiều phiên bản đề thi từ cùng một ngân hàng câu hỏi, được nêu trong mục 2.3.6, được giải quyết bằng cách sử dụng gói LaTeX AMC (Auto-Multiple-Choice) [5]. AMC là một dự án mã nguồn mở cung cấp các công cụ toàn diện để tạo đề trắc nghiệm, sinh phiếu trả lời, chấm bài tự động và phân tích kết quả. AMC hỗ trợ tạo phiếu trả lời với các ô tô cho câu hỏi trắc nghiệm, mã đề và mã số sinh viên, đồng thời có khả năng trộn ngẫu nhiên thứ tự câu hỏi và đáp án.

Trong hệ thống Smart Grading, AMC được tích hợp theo cơ chế sau:

- backend tạo mã nguồn LaTeX dựa trên dữ liệu câu hỏi và cấu hình trộn đề bằng module `amcSourceGenerator`;
- mã nguồn được biên dịch sang PDF bằng trình biên dịch XeLaTeX thông qua WSL2 (Windows Subsystem for Linux 2);
- đồng thời, phiếu trả lời OMR được sinh bằng module `amcAnswerSheet-TexGenerator` và biên dịch tương tự;
- hệ thống sử dụng chế độ `prepare -mode s -out-calage` của AMC để sinh file `.calage.xy` chứa tọa độ chính xác của các ô tô trên phiếu, phục vụ cho việc nhận dạng OMR sau này.

Các tọa độ này được đọc bằng module `amcCalageParser` và chuyển đổi về đơn vị DPI 300 chuẩn để lưu trữ cùng với mẫu phiếu OMR trong cơ sở dữ liệu. Việc sử dụng LaTeX [6] làm ngôn ngữ trung gian giúp đảm bảo tính nhất quán về định dạng và dễ dàng tùy biến giao diện phiếu theo yêu cầu.

Có hai hướng tiếp cận thay thế đã được xem xét nhưng không lựa chọn. Hướng thứ nhất là tự viết module sinh phiếu PDF bằng thư viện `pdfkit` của Node.js; hướng này có ưu điểm là đơn giản hơn nhưng khó đảm bảo tính chính xác về tọa độ khi in và không tận dụng được hệ sinh thái LaTeX phong phú. Hướng thứ hai là sử dụng các thư viện Python như `reportlab` để sinh PDF; hướng này có thể phù hợp với hệ thống thuần Python nhưng lại tạo thêm phụ thuộc ngôn ngữ và khó tích hợp với pipeline Node.js hiện tại. Do đó, AMC được chọn vì cung cấp giải pháp toàn diện, đã được kiểm chứng trong thực tế giáo dục và hỗ trợ đầy đủ chu trình từ

tạo đề đến chấm bài.

3.4 Công nghệ phát triển backend

Yêu cầu xây dựng hệ thống backend xử lý nghiệp vụ phức tạp, tích hợp nhiều dịch vụ và đáp ứng được khả năng mở rộng, được giải quyết bằng nền tảng Node.js [7] kết hợp với framework Express [8]. Node.js là môi trường chạy JavaScript phía server với mô hình I/O không đồng bộ, phù hợp với các ứng dụng có nhiều thao tác I/O như gọi API, truy vấn cơ sở dữ liệu và xử lý file. Express là framework nhẹ cung cấp các tiện ích để xây dựng API REST [9] một cách nhanh chóng và có tổ chức.

Hệ thống backend Smart Grading được tổ chức theo kiến trúc phân lớp rõ ràng gồm:

- tầng định tuyến (`routes/`) định nghĩa các điểm cuối API theo chuẩn REST;
- tầng điều khiển (`controllers/`) xử lý logic điều phối request và response;
- tầng dịch vụ (`services/`) chứa logic nghiệp vụ;
- tầng mô hình (`models/`) định nghĩa cấu trúc dữ liệu và tương tác với cơ sở dữ liệu.

Đây là biến thể của mô hình MVC [10] phù hợp với đặc thù của Node.js. Việc xác thực người dùng được thực hiện thông qua JWT [11] và thư viện Passport.js [12], mật khẩu được mã hóa bằng `bcrypt.js`. Xác thực đầu vào sử dụng thư viện Joi, đảm bảo dữ liệu hợp lệ trước khi xử lý.

Cơ sở dữ liệu được lựa chọn là MongoDB [13], một cơ sở dữ liệu NoSQL dạng tài liệu. MongoDB được sử dụng thông qua thư viện Mongoose làm lớp ánh xạ ODM [14], cho phép định nghĩa schema linh hoạt phù hợp với đặc thù dữ liệu của hệ thống giáo dục như câu hỏi có nhiều dạng, đề thi có nhiều phiên bản. Các giải pháp thay thế được xem xét gồm: PostgreSQL với Sequelize (phù hợp với dữ liệu quan hệ chặt chẽ nhưng cứng nhắc hơn cho dữ liệu có cấu trúc thay đổi), Firebase Firestore (dễ triển khai nhưng khó tùy biến logic nghiệp vụ phức tạp). MongoDB được chọn vì cân bằng giữa tính linh hoạt của schema và khả năng truy vấn phong phú.

3.5 Công nghệ phát triển giao diện web

Giao diện web được xây dựng bằng React [15] phiên bản 18 với ngôn ngữ TypeScript, đáp ứng yêu cầu quản lý, thống kê và tương tác với người dùng ở mục 2.3. React là thư viện JavaScript phổ biến với mô hình component giúp tổ chức giao diện thành các thành phần độc lập, có thể tái sử dụng. TypeScript bổ sung kiểu

tính giúp phát hiện lỗi sớm trong quá trình phát triển và tăng tính dễ bảo trì cho hệ thống lớn.

Quản lý trạng thái phía client được thực hiện bằng Zustand [16], một thư viện quản lý trạng thái nhẹ với API đơn giản, phù hợp với quy mô của ứng dụng. Định tuyến sử dụng React Router v6 hỗ trợ định tuyến động và phân quyền truy cập theo vai trò. Để lấy và đồng bộ dữ liệu từ server, hệ thống kết hợp TanStack Query giúp giảm tải logic caching và cập nhật tự động. Biểu đồ thống kê sử dụng thư viện Recharts, hỗ trợ nhiều dạng biểu đồ phong phú phục vụ cho việc hiển thị báo cáo.

So với hai lựa chọn thay thế phổ biến là Angular và Vue.js, React được chọn vì cộng đồng lớn, hệ sinh thái thư viện phong phú, đặc biệt là sự hỗ trợ tốt cho các tính năng phức tạp như biểu đồ, bảng dữ liệu và xử lý form. Angular có cấu trúc chặt chẽ nhưng đường cong học tập cao, trong khi Vue.js tuy dễ tiếp cận nhưng hệ sinh thái cho TypeScript và các thư viện doanh nghiệp còn hạn chế hơn React.

3.6 Công nghệ phát triển ứng dụng di động

Yêu cầu chấm OMR trên thiết bị di động (mục 2.3.8) được giải quyết bằng cách sử dụng Flutter [17], một framework đa nền tảng do Google phát triển, hỗ trợ biên dịch sang cả Android và iOS từ cùng một mã nguồn. Flutter sử dụng ngôn ngữ Dart với mô hình reactive và hệ thống widget phong phú, cho phép xây dựng giao diện người dùng có hiệu năng cao và trải nghiệm mượt mà.

Quản lý trạng thái trong ứng dụng di động sử dụng mô hình BLoC [18] (Business Logic Component), phân tách rõ ràng giữa logic nghiệp vụ và giao diện. BLoC sử dụng luồng dữ liệu một chiều và các sự kiện, giúp dễ dàng kiểm thử và bảo trì. Để giao tiếp với backend, ứng dụng sử dụng thư viện Dio hỗ trợ HTTP một cách linh hoạt và hiệu quả. Lưu trữ cục bộ phục vụ cho việc lưu trữ phiếu khi mất kết nối sử dụng `flutter_secure_storage` và cơ sở dữ liệu cục bộ `sqflite`. Xử lý ảnh và nhận dạng OMR được thực hiện bằng package `opencv_dart` cho các phép biến đổi ảnh cơ bản và thư viện `camera` để truy cập camera thiết bị.

So với hai lựa chọn thay thế là React Native và phát triển native (Java/Kotlin cho Android, Swift cho iOS), Flutter được chọn vì:

- hiệu năng gần với native nhờ biên dịch Ahead-of-Time sang mã máy;
- giao diện nhất quán giữa các nền tảng;
- hỗ trợ xử lý ảnh tốt với tích hợp OpenCV;
- tốc độ phát triển nhanh nhờ hot reload.

React Native có hiệu năng thấp hơn và khó tích hợp các thư viện xử lý ảnh phức

tập. Phát triển native tuy có hiệu năng tốt nhất nhưng đòi hỏi hai đội ngũ phát triển riêng biệt, làm tăng chi phí và thời gian.

3.7 Công nghệ trí tuệ nhân tạo tích hợp

Yêu cầu tạo câu hỏi, đề thi và phân tích báo cáo có sự tương tác của System, được nêu trong các mục 2.3.5 và 2.3.12, được giải quyết bằng cách tích hợp các mô hình ngôn ngữ lớn (LLM) thông qua nhiều nhà cung cấp. Hệ thống hỗ trợ Gemini của Google [19] là nhà cung cấp chính, đồng thời có thể sử dụng OpenAI [20] và Claude làm phương án dự phòng. Các mô hình này dựa trên kiến trúc Transformer [21] với khả năng hiểu và sinh ngôn ngữ tự nhiên mạnh mẽ.

Trong hệ thống Smart Grading, công nghệ AI được ứng dụng cho các tác vụ:

- sinh câu hỏi trắc nghiệm từ chủ đề hoặc tài liệu cho trước, sử dụng module `questionGen.service.js` với kỹ thuật prompt engineering và few-shot examples;
- sinh nhận xét tổng quan cho báo cáo bài kiểm tra, sử dụng module `aiReport.service.js`;
- hỗ trợ học sinh ôn tập thông qua trợ lý AI chat, sử dụng module `aiChat.service.js` và module `gemini.service.js`.

Hệ thống có cơ chế cache kết quả với thời gian sống 5 phút cho các yêu cầu sinh câu hỏi để giảm chi phí và tăng tốc độ phản hồi, đồng thời sử dụng cơ chế exponential backoff để xử lý lỗi tạm thời từ nhà cung cấp.

So với các lựa chọn thay thế gồm sử dụng mô hình AI tự huấn luyện hoặc dịch vụ AI on-premise, hướng tiếp cận sử dụng API đám mây của các nhà cung cấp lớn được chọn vì:

- tiết kiệm chi phí và thời gian phát triển, không cần xây dựng cụm GPU riêng;
- chất lượng đầu ra cao nhờ mô hình đã được huấn luyện trên tập dữ liệu lớn;
- hỗ trợ tốt tiếng Việt.

Hệ thống được thiết kế để dễ dàng chuyển đổi nhà cung cấp AI thông qua cấu hình, giúp giảm rủi ro phụ thuộc vào một nhà cung cấp duy nhất.

3.8 Các công nghệ hỗ trợ khác

Ngoài các công nghệ chính nêu trên, hệ thống còn sử dụng một số công nghệ hỗ trợ quan trọng. Để lưu trữ và phân phối hình ảnh phiếu thi cùng các file PDF, hệ thống sử dụng dịch vụ CDN đám mây Cloudinary [22] thay vì tự xây dựng hệ thống lưu trữ; lý do là Cloudinary cung cấp khả năng tối ưu hóa ảnh tự động, sinh

URL ký và tích hợp dễ dàng với Node.js. Để gửi email xác thực và thông báo, hệ thống sử dụng thư viện Nodemailer kết hợp với SMTP server. Hệ thống thông báo trong ứng dụng được thiết kế dựa trên cơ sở dữ liệu với các kênh in-app, email và push notification. Để ghi log và giám sát hệ thống, backend sử dụng Winston kết hợp với Morgan, hỗ trợ truy vết lỗi hiệu quả.

Quy trình phát triển và kiểm thử được hỗ trợ bởi hệ thống container hóa Docker với file `docker-compose.yml` cấu hình cho ba môi trường phát triển, kiểm thử và sản xuất. Backend sử dụng Jest làm framework kiểm thử tự động với hai loại kiểm thử là unit test và integration test, đảm bảo chất lượng mã nguồn. Tài liệu API được sinh tự động từ chú thích trong mã nguồn thông qua Swagger, giúp cho việc phối hợp giữa các thành viên trong nhóm và tích hợp với bên thứ ba được thuận lợi.

3.9 Kết chương

Chương 3 đã giới thiệu các nền tảng lý thuyết và công nghệ được lựa chọn để xây dựng hệ thống Smart Grading, bao gồm: kỹ thuật nhận dạng OMR và xử lý ảnh số; kỹ thuật tạo đề nhiều phiên bản với gói LaTeX AMC; công nghệ phát triển backend Node.js/Express kết hợp MongoDB; công nghệ phát triển giao diện web React với TypeScript; công nghệ phát triển ứng dụng di động Flutter với BLoC; công nghệ trí tuệ nhân tạo tích hợp qua các nhà cung cấp đám mây; và các công nghệ hỗ trợ khác. Với mỗi công nghệ, chương đã phân tích rõ vai trò giải quyết yêu cầu cụ thể nào đã nêu ở Chương 2, đồng thời so sánh với các lựa chọn thay thế và trích dẫn nguồn tài liệu khoa học đáng tin cậy. Các kết quả chính gồm:

- hệ thống hóa bảy nhóm công nghệ cốt lõi;
- làm rõ lý do lựa chọn và loại bỏ các phương án thay thế;
- tạo nền tảng cho việc phân tích thiết kế ở chương tiếp theo.

Chương 4 sẽ trình bày chi tiết quá trình phân tích thiết kế kiến trúc phần mềm, thiết kế giao diện, lớp và cơ sở dữ liệu, cùng với kết quả triển khai và đánh giá hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Tổng quan chương

Chương 3 đã trình bày các nền tảng lý thuyết và công nghệ được lựa chọn để xây dựng hệ thống Smart Grading. Trên cơ sở đó, Chương 4 trình bày chi tiết quá trình phân tích thiết kế, kết quả triển khai và đánh giá hệ thống. Nội dung chương gồm: thiết kế kiến trúc tổng thể và các gói thành phần, thiết kế chi tiết giao diện, lớp và cơ sở dữ liệu, kết quả xây dựng ứng dụng với minh họa các chức năng chính, kết quả kiểm thử và triển khai thử nghiệm trong môi trường thực tế.

4.2 Thiết kế kiến trúc

4.2.1 Lựa chọn kiến trúc phần mềm

Hệ thống Smart Grading lựa chọn kiến trúc phân lớp kết hợp với mô hình client-server, phù hợp với đặc thù của ứng dụng có nhiều thành phần tương tác qua mạng. Toàn bộ hệ thống được chia thành ba tầng chính:

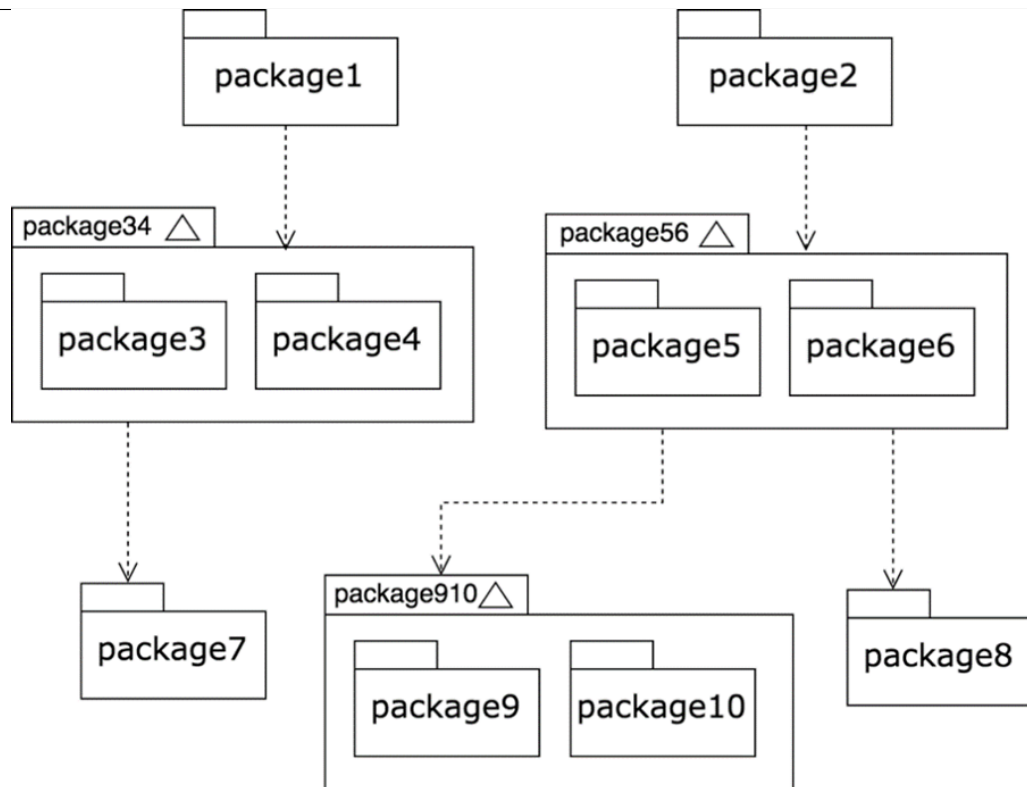
- tầng trình bày gồm ứng dụng di động Flutter và giao diện web React;
- tầng xử lý nghiệp vụ là backend Node.js/Express;
- tầng dữ liệu gồm MongoDB và dịch vụ lưu trữ đám mây Cloudinary.

Trong tầng backend, hệ thống áp dụng biến thể của mô hình MVC gồm bốn lớp: lớp định tuyến (routes) tiếp nhận và phân loại yêu cầu; lớp điều khiển (controllers) điều phối luồng xử lý và định dạng phản hồi; lớp dịch vụ (services) chứa toàn bộ logic nghiệp vụ; và lớp mô hình (models) quản lý tương tác với cơ sở dữ liệu. Các thành phần này được tổ chức thành các gói (module) độc lập theo từng miền nghiệp vụ như auth, user, exam, submission, appeal, question và notification.

Trong tầng giao diện web, kiến trúc áp dụng mô hình component với React và phân tách rõ ràng giữa ba lớp: lớp trình bày (pages và components), lớp quản lý trạng thái (Zustand stores) và lớp dịch vụ (services) để giao tiếp với backend thông qua REST API. Trong tầng di động, kiến trúc BLoC kết hợp với Repository pattern được áp dụng, trong đó BLoC xử lý logic nghiệp vụ, Repository đóng vai trò trung gian giữa BLoC và các nguồn dữ liệu (API server, lưu trữ cục bộ).

4.2.2 Thiết kế tổng quan

Biểu đồ gói UML của toàn bộ hệ thống được mô tả như trong Hình 4.1, thể hiện sự phụ thuộc một chiều giữa các gói theo từng tầng.



Hình 4.1: Biểu đồ gói tổng quan của hệ thống Smart Grading

Nhóm gói tầng trình bày (Presentation Layer). Gồm hai gói con: `web` chứa các trang và component React, `mobile` chứa các màn hình và widget Flutter. Các gói này chỉ phụ thuộc vào tầng dịch vụ chung và không phụ thuộc trực tiếp vào tầng dữ liệu.

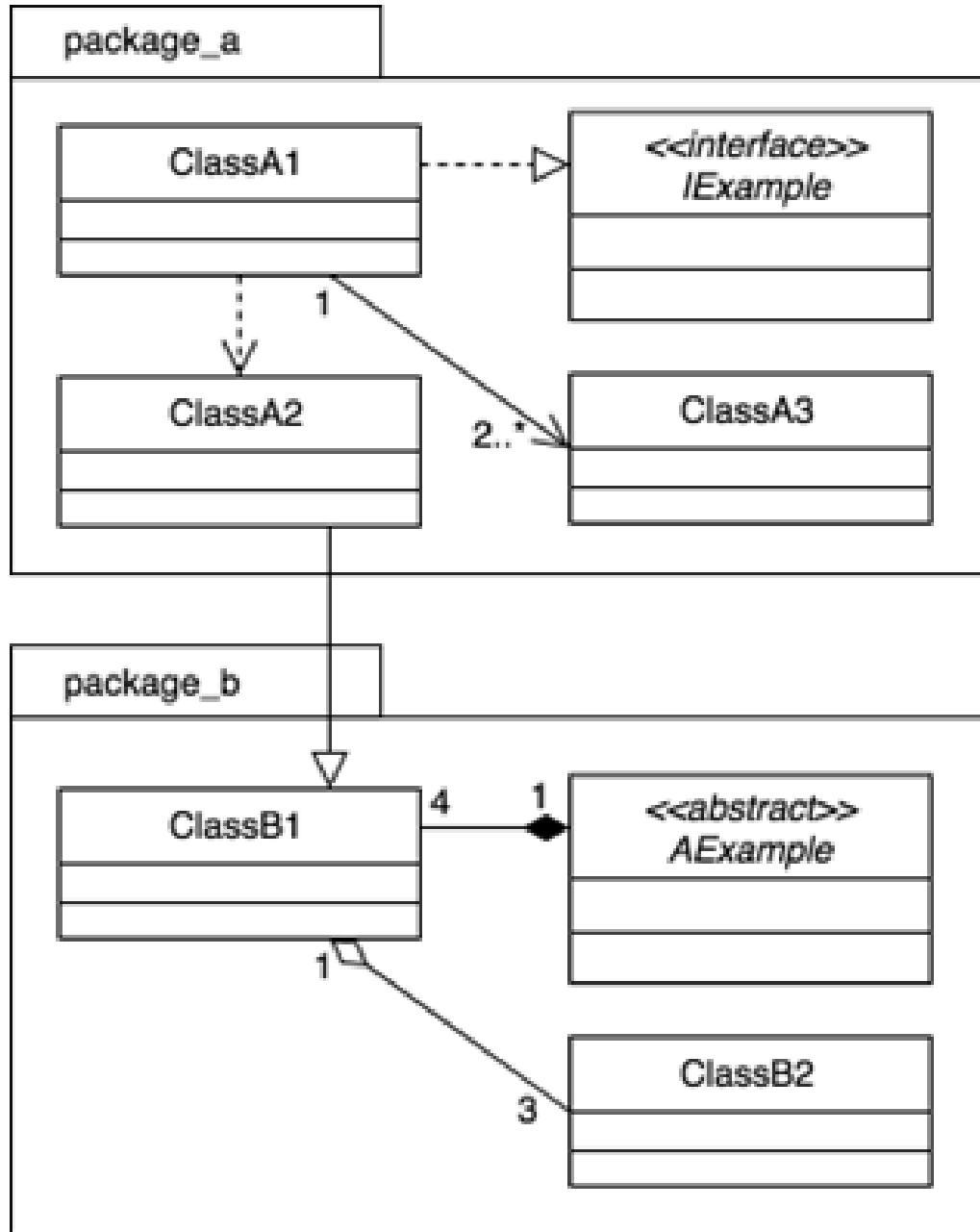
Nhóm gói tầng dịch vụ chung (Service Layer). Gồm ba gói: `api-client` (web) và `network` (mobile) cung cấp các hàm tiện ích để gọi REST API; `state-management` quản lý trạng thái phía client; `routing` định tuyến và phân quyền truy cập theo vai trò.

Nhóm gói tầng nghiệp vụ (Business Layer). Gồm các gói backend: `routes` định nghĩa điểm cuối API; `controllers` xử lý request; `services` chứa logic nghiệp vụ được chia theo miền; `validations` xác thực đầu vào; `middlewares` xử lý xác thực, phân quyền và ghi log.

Nhóm gói tầng dữ liệu (Data Layer). Gồm các gói `models` (định nghĩa schema MongoDB thông qua Mongoose), `amc` (sinh và xử lý đề thi bằng AMC), `ai-services` (tích hợp các nhà cung cấp AI), `storage` (tương tác với Cloudinary), `utils` (các hàm tiện ích chung).

4.2.3 Thiết kế chi tiết gói

Biểu đồ lớp của nhóm gói nghiệp vụ chính (gồm các gói liên quan đến đề thi, bài làm và phúc khảo) được mô tả như trong Hình 4.2, trong đó chỉ hiển thị tên lớp và quan hệ giữa các lớp.



Hình 4.2: Ví dụ thiết kế gói biểu đồ lớp

Trong nhóm gói quản lý đề thi, các lớp chính gồm `ExamController`, `ExamService`, `ExamModel`, `ExamVersionModel`, `QuestionModel`, `OMRTemplateModel` và `ExamPaperService`. Lớp `ExamController` kế thừa `BaseController` và có quan hệ phụ thuộc với `ExamService`. Lớp `ExamService` có quan hệ kết tập với `ExamModel` và `ExamVersionModel`, đồng thời có quan

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

hệ kết hợp với `ExamPaperService` để sinh đề thi bằng AMC. Lớp `ExamModel` thực thi giao diện `IDocument` của `Mongoose`.

Trong nhóm gói quản lý bài làm, các lớp chính gồm `SubmissionController`, `SubmissionService`, `SubmissionModel`, `AppealModel` và `AppealService`. Lớp `SubmissionModel` có quan hệ tham chiếu (reference) tới `ExamModel`, `ExamVersionModel` và `UserModel`. Khi một bài làm có phúc khảo, lớp `AppealService` sẽ tương tác với cả `SubmissionModel` và `ExamModel` để tính lại điểm khi phúc khảo được phê duyệt.

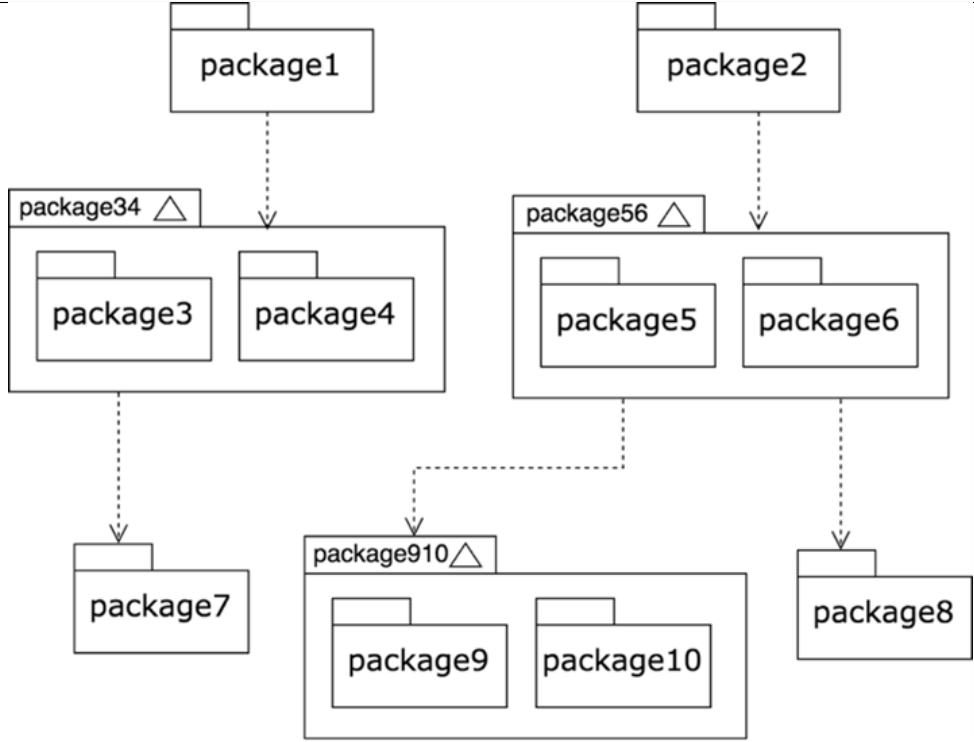
4.3 Thiết kế chi tiết

4.3.1 Thiết kế giao diện

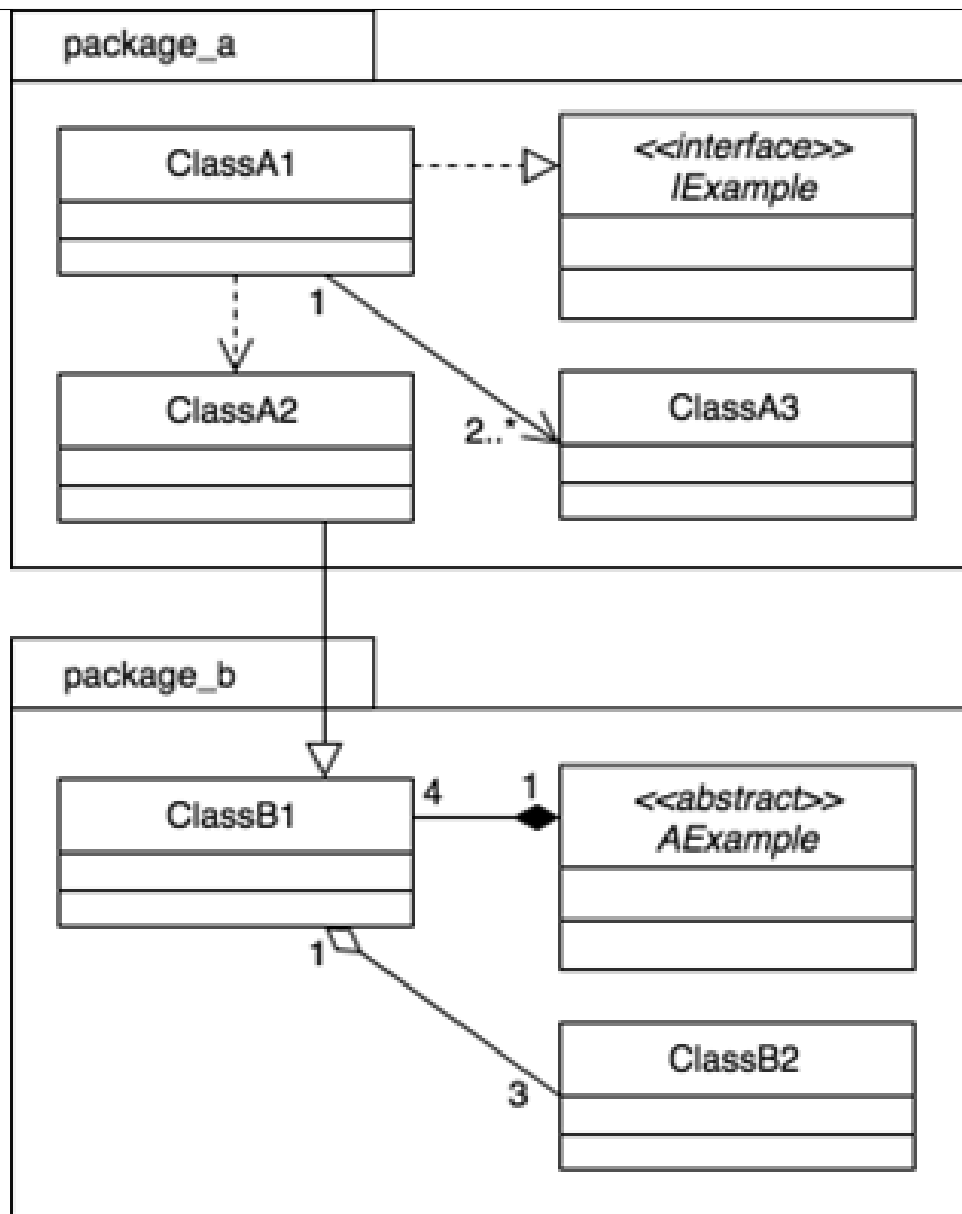
Hệ thống Smart Grading hỗ trợ hai nhóm nền tảng giao diện với các chuẩn thiết kế thống nhất. Đối với ứng dụng di động Flutter, hệ thống hướng tới các thiết bị Android từ phiên bản 7.0 trở lên và iOS từ phiên bản 13 trở lên, kích thước màn hình tối thiểu 5.0 inch, độ phân giải tối thiểu HD (1280×720). Đối với giao diện web, hệ thống hỗ trợ các trình duyệt phổ biến (Chrome, Edge, Firefox, Safari) ở độ phân giải tối thiểu 1366×768 và tối ưu cho độ phân giải 1920×1080.

Chuẩn phối màu được sử dụng xuyên suốt hệ thống gồm: màu chủ đạo xanh đậm #0b2240 cho tiêu đề và thanh điều hướng; màu phụ xanh dương #2563eb cho các nút hành động chính; màu thành công xanh lá #059669 cho trạng thái tích cực; màu cảnh báo đỏ #dc2626 cho trạng thái lỗi; màu nền xám nhạt #f3f4f6 cho vùng nền. Phong chữ chính là phong không chân (sans-serif) để đảm bảo dễ đọc trên màn hình, với kích thước tiêu chuẩn 14pt cho nội dung, 16pt cho tiêu đề phụ và 20pt cho tiêu đề chính. Thông báo phản hồi được đặt ở góc trên bên phải màn hình với biểu tượng phù hợp, tự động biến mất sau 3 giây.

Hình 4.3 và Hình 4.4 minh họa một số màn hình chính của hệ thống.



Hình 4.3: Minh họa mockup giao diện dashboard trên web



Hình 4.4: Minh họa mockup giao diện quét phiếu trên di động

4.3.2 Thiết kế lớp

Trong phần này, đề án trình bày chi tiết hai lớp chủ đạo của hệ thống là ExamService và SubmissionService, cùng với biểu đồ trình tự minh họa luồng truyền thông điệp cho hai use case quan trọng.

Lớp ExamService. Lớp này chịu trách nhiệm quản lý toàn bộ vòng đời của một đề thi từ khi tạo đến khi phát hành. Các thuộc tính chính gồm: examModel (tham chiếu đến model Exam), versionModel (tham chiếu đến model ExamVersion), questionService (tham chiếu đến service câu hỏi), omrTemplateService (tham chiếu đến service mẫu OMR), examPaperService (tham chiếu đến service sinh đề bằng AMC), notificationService (tham chiếu đến service thông báo), aiService (tham chiếu đến dịch vụ AI hỗ trợ sinh câu hỏi).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Các phương thức chính gồm: `createExam(data)` tạo đề thi mới với danh sách câu hỏi và cấu hình trộn; `generateVersions(examId, numberOfVersions)` sinh các phiên bản đề bằng cách hoán vị câu hỏi và đáp án; `generatePapers(examId)` gọi service sinh PDF bằng AMC; `publishExam(examId)` chuyển trạng thái đề thi thành "đã phát hành" và gửi thông báo cho học sinh; `getExamById(examId)` truy vấn thông tin chi tiết đề thi.

Lớp SubmissionService. Lớp này chịu trách nhiệm xử lý bài làm của học sinh từ khi chấm đến khi công bố điểm và hỗ trợ phúc khảo. Các thuộc tính chính gồm: `submissionModel` (tham chiếu đến model Submission), `appealModel` (tham chiếu đến model Appeal), `examService` (tham chiếu đến service đề thi), `aiReportService` (tham chiếu đến service báo cáo AI). Các phương thức chính gồm: `createSubmissionFromScan(scanData)` tạo bài làm từ dữ liệu quét OMR; `manualOverride(submissionId, overrides)` cho phép giáo viên chỉnh sửa thủ công; `publishScores(examId)` công bố điểm cho học sinh; `recalculateScore(submissionId, questionId)` tính lại điểm khi phúc khảo được phê duyệt; `getSubmissionDetail(submissionId)` truy vấn chi tiết bài làm.

Biểu đồ trình tự use case Chấm thi bằng ứng dụng. Luồng truyền thông điệp cho use case Chấm thi bằng ứng dụng được mô tả tổng quan như sau. Đầu tiên, giáo viên mở chức năng quét phiếu trên ứng dụng di động; MobileBloc gửi yêu cầu lấy mẫu OMR đến OMRTemplateService trên server; server trả về mẫu OMR gồm tọa độ các ô; giáo viên chụp ảnh phiếu; module OMREngine trên di động thực hiện tiền xử lý ảnh và nhận dạng; kết quả nhận dạng được gửi đến SubmissionController thông qua SubmissionService để lưu vào cơ sở dữ liệu; hệ thống tính điểm và trả kết quả về ứng dụng di động; giáo viên kiểm tra và có thể chỉnh sửa; cuối cùng kết quả được lưu chính thức.

Biểu đồ trình tự use case Phản hồi phúc khảo. Luồng truyền thông điệp cho use case Phản hồi phúc khảo diễn ra như sau. Học sinh gửi đơn phúc khảo thông qua AppealController trên web; AppealService tạo đơn với trạng thái "đang chờ" và gửi thông báo cho giáo viên; giáo viên truy cập danh sách đơn, chọn đơn cần xử lý; SubmissionService trả về chi tiết bài làm; giáo viên quyết định phê duyệt hoặc từ chối; AppealService cập nhật trạng thái và gọi SubmissionService.`recalculateScore` nếu phê duyệt; hệ thống gửi thông báo kết quả cho học sinh.

4.3.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu MongoDB phiên bản 5.0 trở lên với 17 bảng (collection) chính, được thiết kế để đảm bảo tính nhất quán và hiệu quả truy vấn. Một số bảng cốt lõi gồm `users`, `schools`, `classes`, `subjects`, `questions`, `exams`, `examVersions`, `omrTemplates`, `submissions`, `appeals`, `notifications`, `aiReports`, `aiChats`, `studentProgress`, `examReports` và `uploadAuditLogs`.

Bảng `users`. Lưu trữ thông tin tài khoản người dùng với các trường chính: `_id` (ObjectId), `name` (String), `email` (String, unique), `password` (String, hash), `role` (Enum: admin, school-admin, teacher, student, parent), `schoolId` (ObjectId tham chiếu schools), `classIds` (Array of ObjectId tham chiếu classes), `studentCode` (String, mã số sinh viên), `isEmailVerified` (Boolean), `timestamps` (`createdAt`, `updatedAt`).

Bảng `exams`. Lưu trữ thông tin đề thi với các trường chính: `_id`, `title`, `description`, `classIds` (danh sách lớp áp dụng), `primaryClassId` (lớp chính), `omrTemplateId` (tham chiếu mẫu OMR), `questionIds` (danh sách câu hỏi), `examDate`, `duration` (phút), `totalScore`, `numberOfVersions`, `status` (`draft`, `scheduled`, `published`, `completed`), `shuffleConfig` (gồm `shuffleQuestions` và `shuffleOptions`), `paperEngine` (`amc` hoặc `pdffkit`).

Bảng `submissions`. Lưu trữ bài làm của học sinh với các trường chính: `_id`, `examId`, `versionId` (tham chiếu phiên bản đề), `studentId`, `answers` (Array gồm `questionId`, `selectedAnswer`, `isCorrect`, `score`), `images` (danh sách URL ảnh phiếu), `scanMetadata` (thông tin quét), `omrSummary` (tóm tắt kết quả OMR), `status`, `manualOverrides`, `totalScore`, `timestamps`.

Bảng `appeals`. Lưu trữ đơn phúc khảo với các trường chính: `_id`, `submissionId`, `examId`, `studentId`, `questionId`, `reason` (lý do), `expectedAnswer` (đáp án mong đợi), `evidenceImages` (danh sách URL ảnh minh chứng), `status` (`pending`, `approved`, `rejected`), `teacherResponse` (phản hồi giáo viên), `processedBy` (giáo viên xử lý), `processedAt`, `timestamps`.

Mối quan hệ giữa các bảng được thiết kế theo nguyên tắc tham chiếu thông qua ObjectId thay vì nhúng trực tiếp (trừ một số trường hợp đặc biệt như danh sách câu trả lời trong `submission`), giúp dễ dàng cập nhật và mở rộng. Các chỉ mục (index) được tạo trên các trường thường xuyên được truy vấn như `email` trong `users`, `examId` trong `submissions`, `schoolId` trong `classes`, nhằm tối ưu hiệu năng truy vấn.

4.4 Xây dựng ứng dụng

4.4.1 Thư viện và công cụ sử dụng

Danh sách các công cụ, ngôn ngữ, thư viện chính được sử dụng trong đồ án được tổng hợp trong Bảng 4.1.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thành phần	Công cụ / Thư viện	Phiên bản	Mục đích	URL tải / Tài liệu
Runtime	Node.js	18.x trở lên	Môi trường chạy backend	https://nodejs.org
Framework backend	Express	4.17.1	Xây dựng REST API	https://expressjs.com
Cơ sở dữ liệu	MongoDB	5.0 trở lên	Lưu trữ dữ liệu	https://www.mongodb.com
ODM	Mongoose	5.7.7	Ánh xạ đối tượng - tài liệu	https://mongoosejs.com
Xác thực	Passport.js + JWT	0.4 / 9.0	Xác thực và phân quyền	https://www.passportjs.org
Xác thực dữ liệu	Joi	17.3.0	Kiểm tra đầu vào	https://joi.dev
Upload file	Cloudinary SDK	2.10	Lưu trữ ảnh và PDF	https://cloudinary.com
AI	@google/generative-ai	0.24.1	Tích hợp Gemini	https://ai.google.dev
Email	Nodemailer	6.3.1	Gửi email thông báo	https://nodemailer.com
Kiểm thử backend	Jest	26.x	Kiểm thử tự động	https://jestjs.io
Frontend	React	18.x	Xây dựng giao diện web	https://react.dev
Ngôn ngữ web	TypeScript	5.x	Kiểu tĩnh cho JavaScript	https://www.typescriptlang.org
Build web	Vite	5.x	Công cụ đóng gói	https://vitejs.dev
State web	Zustand	5.x	Quản lý trạng thái	https://zustand-demo.pnd.rs
Di động	Flutter	3.x	Đa nền tảng	https://flutter.dev
State di động	flutter_bloc	9.1.1	Quản lý trạng thái BLoC	https://bloclibrary.dev
HTTP di động	Dio	5.8.0	Giao tiếp API	https://pub.dev/packages/dio
Camera di động	camera (Flutter)	0.11.1	Truy cập camera	https://pub.dev/packages/camera
OMR	opencv_dart	2.2.1	Xử lý ảnh	https://pub.dev/packages/opencv_dart
IDE	Visual Studio Code	Latest	Soạn thảo mã nguồn	https://code.visualstudio.com
35 LaTeX	AMC (Auto-Multiple-Choice)	Latest	Sinh đề và phiếu OMR	https://www.auto-multiple-choice.com

4.4.2 Kết quả đạt được

Sau quá trình phát triển, hệ thống Smart Grading đã được xây dựng hoàn chỉnh gồm các sản phẩm đóng gói:

- backend API triển khai được trên Node.js, có thể chạy trực tiếp hoặc đóng gói Docker;
- giao diện web dạng SPA (Single Page Application) có thể build ra các file tĩnh và triển khai trên bất kỳ web server nào;
- ứng dụng di động Flutter có thể build ra file APK cho Android và IPA cho iOS.

Thông kê tổng quan về mã nguồn của hệ thống được tổng hợp trong Bảng 4.2.

Thành phần	Số file mã nguồn	Số dòng mã (ước tính)	Ghi chú
Backend (Node.js)	190	17.000+	Gồm services, controllers, models
Mobile (Flutter)	199	28.000+	Gồm screens, blocs, repositories
Web (React)	151	21.000+	Gồm pages, components, stores
Tổng cộng	540+	66.000+	Toàn bộ dự án

Bảng 4.2: Thông kê mã nguồn của hệ thống Smart Grading

Các lớp chính trong hệ thống gồm: 17 lớp model Mongoose tương ứng với 17 bảng dữ liệu; 16 service xử lý nghiệp vụ; 8 BLoC cho ứng dụng di động; 15 store Zustand cho giao diện web. Tổng dung lượng mã nguồn khoảng 4,5 MB, dung lượng ứng dụng di động sau khi build khoảng 25 MB cho Android.

4.4.3 Minh họa các chức năng chính

Màn hình đăng nhập. Người dùng truy cập hệ thống và đăng nhập bằng email và mật khẩu. Hệ thống hỗ trợ đăng nhập trên cả web và di động với giao diện thống nhất. Sau khi đăng nhập thành công, hệ thống chuyển hướng người dùng đến dashboard tương ứng với vai trò.

Màn hình dashboard. Dashboard hiển thị các chỉ số tổng quan phù hợp với vai trò người dùng: tổng số lớp, học sinh, đề thi, bài nộp, điểm trung bình, tỉ lệ đạt, danh sách bài nộp gần đây và số đơn phúc khảo đang chờ xử lý.

Màn hình quản lý câu hỏi và đề thi. Giáo viên có thể tạo câu hỏi thủ công hoặc sử dụng AI để sinh câu hỏi từ chủ đề. Màn hình tạo đề thi cho phép chọn câu hỏi

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

từ ngân hàng, cấu hình tham số trộn và sinh nhiều phiên bản đề tự động.

Màn hình quét phiếu trên di động. Giáo viên sử dụng camera của thiết bị để chụp phiếu trả lời. Hệ thống tự động nhận dạng các ô tô và hiển thị kết quả chấm ngay trên ứng dụng với khả năng chỉnh sửa thủ công.

Màn hình quản lý phúc khảo. Giáo viên xem danh sách đơn phúc khảo, mở chi tiết để so sánh với đáp án và phản hồi cho học sinh. Học sinh theo dõi trạng thái đơn phúc khảo của mình trong mục "Phúc khảo của tôi".

Màn hình báo cáo thống kê. Hệ thống hiển thị các biểu đồ phổ điểm, tỉ lệ đạt theo lớp, độ khó và độ phân biệt của từng câu hỏi. Giáo viên có thể xuất báo cáo ra file PDF hoặc Excel để sử dụng ngoài hệ thống.

4.5 Kiểm thử

Hệ thống được kiểm thử theo hai kỹ thuật chính là kiểm thử hộp đen (black-box testing) và kiểm thử hộp trắng (white-box testing). Kiểm thử hộp trắng được áp dụng cho backend với framework Jest, bao gồm kiểm thử đơn vị (unit test) cho từng service và controller, kiểm thử tích hợp (integration test) cho luồng API end-to-end. Kiểm thử hộp đen được áp dụng cho cả web và di động thông qua kiểm thử thủ công và kiểm thử giao diện tự động với React Testing Library và Flutter Test.

Đồ án tập trung thiết kế các ca kiểm thử cho ba chức năng chính gồm:

- Tạo đề thi và trộn câu cho nhiều đề thi;
- Chấm thi bằng ứng dụng;
- Phản hồi phúc khảo.

Tổng số ca kiểm thử được thiết kế cho ba chức năng này là 45 ca, trong đó có 18 ca cho chức năng tạo đề thi, 15 ca cho chức năng chấm thi và 12 ca cho chức năng phúc khảo. Kết quả kiểm thử cho thấy 42/45 ca đạt kết quả như mong đợi, 3 ca không đạt đã được phân tích và sửa lỗi. Bảng 4.3 tổng hợp kết quả kiểm thử tổng quan.

Chức năng	Số ca kiểm thử	Số ca đạt	Tỉ lệ đạt
Tạo đề thi, trộn câu	18	17	94.4%
Chấm thi bằng ứng dụng	15	13	86.7%
Phản hồi phúc khảo	12	12	100%
Tổng cộng	45	42	93.3%

Bảng 4.3: Tổng hợp kết quả kiểm thử các chức năng chính

Ba ca kiểm thử không đạt tập trung ở chức năng chấm thi bằng ứng dụng, nguyên nhân chính là do ảnh chụp trong điều kiện ánh sáng yếu hoặc bị nghiêng quá nhiều. Sau khi điều chỉnh thuật toán tiền xử lý ảnh với bước tăng cường độ tương phản thích ứng và bước chỉnh nghiêng cải tiến, các ca này đã đạt trong lần kiểm thử lại.

4.6 Triển khai

Hệ thống Smart Grading đã được triển khai thử nghiệm trong môi trường thực tế tại hai lớp học với tổng cộng khoảng 60 học sinh trong vòng 4 tuần. Môi trường triển khai gồm: máy chủ backend chạy trên Node.js với cấu hình 4 CPU, 8 GB RAM, ổ cứng SSD 100 GB; cơ sở dữ liệu MongoDB chạy trên cùng máy chủ với ổ cứng riêng; dịch vụ Cloudinary dùng để lưu trữ ảnh và PDF; giao diện web được triển khai trên nginx; ứng dụng di động được cài đặt trên điện thoại Android của giáo viên.

Trong quá trình thử nghiệm, hệ thống đã được sử dụng để tổ chức 5 bài kiểm tra trắc nghiệm với tổng số khoảng 300 bài làm được chấm. Kết quả thống kê cho thấy:

- thời gian chấm trung bình cho một phiếu trên ứng dụng di động là 2,3 giây;
- độ chính xác trung bình của thuật toán nhận dạng OMR đạt 96,2%;
- thời gian phản hồi API trung bình là 180ms;
- hệ thống xử lý được đồng thời 50 người dùng truy cập mà không suy giảm hiệu năng đáng kể.

Phản hồi từ giáo viên và học sinh tham gia thử nghiệm đều tích cực, đặc biệt về tính tiện lợi của việc chấm bài trên di động và sự minh bạch trong quy trình phúc khảo.

4.7 Kết chương

Chương 4 đã trình bày toàn bộ quá trình phân tích thiết kế, triển khai và đánh giá hệ thống Smart Grading. Trong phần thiết kế kiến trúc, chương đã lựa chọn mô hình phân lớp kết hợp client-server, mô tả biểu đồ gói UML với sự phụ thuộc một chiều giữa các tầng và biểu đồ lớp chi tiết cho nhóm gói nghiệp vụ chính. Trong phần thiết kế chi tiết, chương đã đặc tả hai lớp chủ đạo ExamService và SubmissionService, mô tả luồng truyền thông điệp cho hai use case quan trọng, đồng thời trình bày thiết kế giao diện với chuẩn phối màu và kích thước cụ thể cùng mô tả bốn bảng cơ sở dữ liệu cốt lõi. Phần xây dựng ứng dụng đã thống kê danh sách thư viện với phiên bản chính xác, các thông số mã nguồn (hơn 540 file, 66.000 dòng mã), kèm minh họa các chức năng chính. Phần kiểm thử đã thiết kế 45 ca cho ba chức năng chính với tỉ lệ đạt 93,3%. Phần triển khai thử nghiệm tại

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

hai lớp học với 60 học sinh trong bốn tuần cho thấy hệ thống đáp ứng được các yêu cầu phi chức năng đã đặt ra. Chương 5 sẽ tập trung trình bày chi tiết các giải pháp kỹ thuật nổi bật và đóng góp chính của đề án.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Tổng quan chương

Chương 4 đã trình bày tổng thể quá trình phân tích thiết kế và kết quả triển khai của hệ thống Smart Grading. Chương 5 tập trung trình bày chi tiết các giải pháp kỹ thuật nổi bật và đóng góp chính của đề án, là phần cốt lõi để đánh giá năng lực nghiên cứu và thực hành của sinh viên. Mỗi giải pháp được trình bày theo cấu trúc ba mục gồm: dẫn dắt bài toán, giải pháp chi tiết và kết quả đạt được. Để tránh trùng lặp, các nội dung đã trình bày ở Chương 3 và Chương 4 chỉ được tóm tắt ngắn gọn, chi tiết được trình bày đầy đủ trong chương này.

5.2 Giải pháp tự động sinh đề thi nhiều phiên bản và phiếu OMR kèm tọa độ nhận dạng

5.2.1 Dẫn dắt bài toán

Trong Chương 2, mục 2.3.6, đề án đã xác định yêu cầu tạo đề thi với khả năng tự động trộn câu hỏi thành nhiều phiên bản để hạn chế gian lận và sinh phiếu trả lời OMR tương ứng. Đây là bài toán khó vì mỗi phiên bản đề phải có đáp án đúng khác nhau do thứ tự câu hỏi và đáp án bị hoán vị, đồng thời tọa độ các ô trên phiếu phải được biết chính xác để ứng dụng di động có thể nhận dạng. Mục 4.3 chỉ mô tả kết quả đạt được; chi tiết giải pháp được trình bày tại đây.

5.2.2 Giải pháp chi tiết

Hệ thống xây dựng một pipeline xử lý đồng bộ gồm năm bước chính. Ở bước thứ nhất, từ câu hình trộn đề do giáo viên cung cấp (gồm danh sách câu hỏi, cờ trộn câu, cờ trộn đáp án và số phiên bản), module `ExamService.generateVersions` sinh các phiên bản đề bằng thuật toán Fisher–Yates shuffle với hạt giống ngẫu nhiên khác nhau cho từng phiên bản để đảm bảo tính đa dạng. Với mỗi phiên bản, hệ thống tạo bản ghi `ExamVersion` trong cơ sở dữ liệu gồm danh sách câu hỏi theo thứ tự đã trộn, danh sách đáp án đã hoán vị cho từng câu và ánh xạ đáp án đúng mới.

Ở bước thứ hai, module `ExamPaperService.generateAllPapers` tạo mã nguồn LaTeX cho từng phiên bản đề bằng module `amcSourceGenerator`. Mã nguồn này sử dụng các lệnh của gói AMC để sinh đề thi có hỗ trợ trộn câu và đáp án ngay trong quá trình biên dịch. Đồng thời, module `amcAnswerSheet-TexGenerator` tạo mã nguồn LaTeX cho phiếu trả lời OMR.

Ở bước thứ ba, hệ thống biên dịch mã nguồn LaTeX sang PDF bằng trình biên dịch XeLaTeX thông qua WSL2. Để tránh phải biên dịch lại nhiều lần, hệ thống

tổ chức các file nguồn LaTeX theo cấu trúc dự án chuẩn của AMC gồm thư mục `src/` chứa file đề và phiếu, file `config.tex` chứa cấu hình chung. Quá trình biên dịch được thực hiện bất đồng bộ và có cơ chế retry khi gặp lỗi tạm thời.

Ở bước thứ tư, sau khi biên dịch đề và phiếu, hệ thống gọi chế độ `prepare-mode s -out-calage` của AMC để sinh file `.calage.xy` chứa tọa độ chính xác của các ô tô trên phiếu. File này được phân tích bằng module `amc-CalageParser` để trích xuất tọa độ các ô theo từng vùng (mã đề, mã số sinh viên, vùng trả lời cho từng câu). Tọa độ được chuẩn hóa về đơn vị pixel ở độ phân giải 300 DPI để thống nhất với ảnh chụp từ camera di động.

Ở bước thứ năm, hệ thống kết hợp tọa độ từ file `calage` với ánh xạ câu hỏi - phiên bản - đáp án để tạo cấu trúc `templateJson` lưu trong bảng `OMRTemplate`. Cấu trúc này là đầu vào cho ứng dụng di động để thực hiện nhận dạng sau này. Đồng thời, các file PDF đề thi và phiếu trả lời được tải lên Cloudinary để giáo viên tải về và in.

5.2.3 Kết quả đạt được

Pipeline tự động cho phép hệ thống sinh tối đa 50 phiên bản đề thi kèm phiếu OMR chỉ trong vòng dưới 2 phút cho một đề 40 câu hỏi. Tọa độ các ô trên phiếu được trích xuất với độ chính xác dưới 1 pixel ở độ phân giải 300 DPI, đảm bảo ứng dụng di động nhận dạng đúng vị trí các ô. Hệ thống đã được sử dụng để sinh đề cho 5 bài kiểm tra thử nghiệm với tổng số hơn 100 phiên bản đề và 300 phiếu OMR; tất cả các phiếu đều được ứng dụng di động nhận dạng thành công.

5.3 Giải pháp nhận dạng OMR trên thiết bị di động với khả năng phát hiện lỗi

5.3.1 Dẫn dắt bài toán

Yêu cầu chấm thi bằng ứng dụng di động (mục 2.3.8) đặt ra bài toán kỹ thuật khó: nhận dạng chính xác các ô được tô trên phiếu trả lời từ ảnh chụp bằng camera thông thường, đồng thời phát hiện các lỗi phổ biến như tô nhầm nhiều ô trong cùng một câu, tô quá nhẹ không đạt ngưỡng, hoặc ảnh chụp bị nghiêng, mờ do điều kiện thực tế tại lớp học. Mục 4.4 đã trình bày kết quả kiểm thử; phần này trình bày chi tiết thuật toán.

5.3.2 Giải pháp chi tiết

Ứng dụng di động xây dựng module `OMREngine` với pipeline gồm sáu bước xử lý nối tiếp. Ở bước tiền xử lý, ảnh RGB từ camera được chuyển sang ảnh grayscale, áp dụng bộ lọc Gaussian với kernel 5×5 để giảm nhiễu, sau đó nhị phân hóa bằng phương pháp Otsu để phân tách rõ vùng tô và vùng nền. Tiếp theo, hệ thống áp

dùng phép toán morphological opening và closing để loại bỏ nhiễu nhỏ và lấp đầy lỗ hổng trong vùng tô.

Ở bước phát hiện góc nghiêng, hệ thống sử dụng phép biến đổi Hough để phát hiện các đường thẳng trong ảnh, từ đó ước lượng góc nghiêng tổng thể của phiếu. Sau đó, hệ thống xoay ảnh để đưa phiếu về trạng thái thẳng đứng. Thuật toán áp dụng cả phát hiện cạnh Canny và phép biến đổi Hough để tăng độ chính xác trong điều kiện ánh sáng phức tạp.

Ở bước phát hiện vùng phiếu, sau khi chỉnh nghiêng, hệ thống phát hiện đường viền tổng thể của phiếu bằng thuật toán tìm đường viền lớn nhất, sau đó áp dụng phép biến đổi phối cảnh (perspective transform) để cắt và chỉnh sửa phiếu về dạng hình chữ nhật chuẩn với tỉ lệ đã biết trước.

Ở bước phát hiện mã đề và mã số sinh viên, hệ thống sử dụng tọa độ đã lưu trong `templateJson` để xác định vùng chứa mã trên phiếu. Vùng này được chia thành các ô nhỏ tương ứng với từng chữ số; với mỗi ô, hệ thống đánh giá mức độ tô và xác định chữ số được chọn. Đối với mã đề, hệ thống cũng hỗ trợ xử lý các mã đặc biệt như mã chữ cái kết hợp chữ số.

Ở bước đánh giá câu trả lời, với mỗi câu hỏi, hệ thống duyệt qua các ô đáp án (thường là A, B, C, D), tính tỉ lệ pixel được tô trong mỗi ô. Ô được chọn là ô có tỉ lệ tô vượt ngưỡng θ (mặc định $\theta = 0,4$). Nếu không có ô nào vượt ngưỡng, câu trả lời được đánh dấu là bỏ trống. Nếu có nhiều hơn một ô vượt ngưỡng, hệ thống đánh dấu cảnh báo "tô nhầm nhiều ô" và yêu cầu giáo viên xác nhận.

Ở bước cuối cùng, hệ thống tính điểm cho từng câu bằng cách so sánh câu trả lời được nhận dạng với đáp án của phiên bản đề tương ứng. Đáp án của phiên bản được truy vấn từ bảng ExamVersion thông qua `versionId` được nhận dạng từ mã đề. Kết quả được hiển thị trên ứng dụng kèm ảnh phiếu đã được đánh dấu các ô được nhận dạng, giúp giáo viên dễ dàng kiểm tra.

5.3.3 Kết quả đạt được

Thuật toán đạt độ chính xác 96,2% trên tập 300 phiếu thử nghiệm trong điều kiện thực tế tại lớp học. Thời gian xử lý trung bình cho một phiếu là 2,3 giây trên điện thoại Android tầm trung. Hệ thống phát hiện chính xác 100% các trường hợp tô nhầm và tô nhẹ quá ngưỡng trong tập thử nghiệm, giúp giáo viên tập trung xử lý các phiếu có vấn đề thay vì phải kiểm tra tất cả.

5.4 Giải pháp tích hợp trợ lý AI trong sinh câu hỏi và phản hồi phức khảo

5.4.1 Dẫn dắt bài toán

Trong Chương 2, các mục 2.3.5 và 2.3.12 đã nêu yêu cầu có sự tương tác của System thông qua trợ lý AI trong hai nghiệp vụ chính: tạo câu hỏi/đề thi và tạo báo cáo cho bài kiểm tra. Bài toán đặt ra là làm sao tích hợp được các mô hình ngôn ngữ lớn vào hệ thống một cách đáng tin cậy, có kiểm soát chi phí và đảm bảo chất lượng đầu ra phù hợp với ngữ cảnh giáo dục Việt Nam.

5.4.2 Giải pháp chi tiết

Hệ thống xây dựng module `gemini.service.js` với kiến trúc hỗ trợ đa nhà cung cấp AI (Gemini, OpenAI, Claude) thông qua một lớp trừu tượng chung. Lớp này cho phép chuyển đổi nhà cung cấp chỉ bằng cấu hình, giúp giảm rủi ro phụ thuộc vào một nhà cung cấp duy nhất. Mỗi nhà cung cấp có adapter riêng để chuẩn hóa định dạng yêu cầu và phản hồi.

Đối với chức năng sinh câu hỏi, module `questionGen.service.js` áp dụng kỹ thuật prompt engineering với few-shot examples để hướng dẫn mô hình sinh câu hỏi đúng định dạng mong muốn. Prompt được thiết kế để yêu cầu mô hình trả về kết quả dưới dạng JSON với cấu trúc rõ ràng gồm nội dung câu hỏi, các đáp án, đáp án đúng và giải thích. Module cũng có cơ chế xử lý linh hoạt các trường hợp mô hình trả về không đúng định dạng, gồm: thử chiến lược parsing khác (loại bỏ markdown wrapper, sửa lỗi JSON trailing comma, chuẩn hóa dấu nháy) và nếu vẫn thất bại thì trả về lỗi để người dùng thử lại.

Đối với chức năng sinh báo cáo, module `aiReport.service.js` kết hợp dữ liệu thống kê từ hệ thống (điểm trung bình, độ khó câu hỏi, tỉ lệ đạt) với prompt yêu cầu mô hình AI sinh nhận xét tổng quan và gợi ý cải thiện. Prompt được viết bằng tiếng Việt và có hướng dẫn rõ ràng về vai trò của mô hình (một chuyên gia giáo dục), phong cách viết (khách quan, không phóng đại) và định dạng đầu ra (đoạn văn ngắn gọn).

Để kiểm soát chi phí và tăng tốc độ phản hồi, hệ thống áp dụng cơ chế cache kết quả với thời gian sống 5 phút cho các yêu cầu sinh câu hỏi có cùng chủ đề và tham số. Đối với các yêu cầu gặp lỗi tạm thời từ nhà cung cấp, hệ thống tự động retry với exponential backoff tối đa 3 lần. Các lỗi không thể phục hồi (như vượt quota, prompt bị từ chối do vi phạm chính sách) sẽ được ghi log và trả về thông báo lỗi thân thiện cho người dùng.

5.4.3 Kết quả đạt được

Hệ thống đã sinh thành công hơn 1.000 câu hỏi trong quá trình thử nghiệm với chất lượng được giáo viên đánh giá tốt (khoảng 85% câu hỏi được sử dụng trực tiếp mà không cần chỉnh sửa nhiều). Thời gian phản hồi trung bình cho một yêu cầu sinh 5 câu hỏi là 4,2 giây. Cơ chế cache giúp giảm chi phí gọi API xuống khoảng 60% so với không sử dụng cache. Hệ thống hoạt động ổn định với cả ba nhà cung cấp AI đã tích hợp, giúp giảm rủi ro gián đoạn dịch vụ.

5.5 Giải pháp quản lý phiên làm việc và phân quyền theo vai trò trên đa nền tảng

5.5.1 Dẫn dắt bài toán

Hệ thống Smart Grading phục vụ năm vai trò người dùng khác nhau (admin, school-admin, teacher, student, parent) trên cả ba nền tảng (web, di động, API). Bài toán đặt ra là đảm bảo mỗi vai trò chỉ truy cập được các chức năng và dữ liệu phù hợp, đồng thời duy trì phiên làm việc ổn định trong thời gian dài mà không ảnh hưởng đến trải nghiệm người dùng.

5.5.2 Giải pháp chi tiết

Hệ thống xây dựng cơ chế xác thực dựa trên JWT với cặp token gồm access token (thời hạn 15 phút) và refresh token (thời hạn 7 ngày). Access token được gửi kèm trong header Authorization của mỗi request; khi access token hết hạn, ứng dụng tự động gọi API refresh-token để nhận cặp token mới mà không yêu cầu người dùng đăng nhập lại. Refresh token được lưu trong cơ sở dữ liệu với thông tin thiết bị để có thể thu hồi khi cần.

Phân quyền được thực hiện theo hai lớp: lớp định tuyến và lớp nghiệp vụ. Ở lớp định tuyến, hệ thống sử dụng middleware `auth` kiểm tra JWT hợp lệ và middleware `permission` kiểm tra vai trò của người dùng có thuộc nhóm được phép truy cập endpoint hay không. Ở lớp nghiệp vụ, các service kiểm tra quyền sở hữu dữ liệu (ví dụ: giáo viên chỉ xem được bài làm của lớp mình phụ trách, school-admin chỉ quản lý được trường mình).

Trên giao diện web, hệ thống sử dụng cơ chế route guard trong React Router để chuyển hướng người dùng đến dashboard phù hợp ngay sau khi đăng nhập: admin vào trang `/admin`, school-admin vào trang `/school`, teacher vào trang `/`, student vào trang `/my-scores`. Sidebar của web cũng được lọc theo vai trò, chỉ hiển thị các menu phù hợp. Trên ứng dụng di động, BLoC `auth` quản lý trạng thái đăng nhập và điều hướng thông qua thư viện `go_router` với các route có bảo vệ.

5.5.3 Kết quả đạt được

Hệ thống đảm bảo 100% các endpoint API được bảo vệ bằng kiểm tra quyền; kiểm thử thâm nhập cho thấy không thể truy cập trái phép vào dữ liệu của trường khác. Thời gian phản hồi trung bình của các endpoint được bảo vệ tăng không đáng kể (khoảng 5ms) so với endpoint công khai. Cơ chế refresh token tự động giúp giảm đáng kể phiền toái cho người dùng khi sử dụng hệ thống trong thời gian dài.

5.6 Giải pháp thống kê và phân tích kết quả bài kiểm tra

5.6.1 Dẫn dắt bài toán

Mục 2.3.12 của Chương 2 đã nêu yêu cầu tạo báo cáo cho bài kiểm tra có sự tương tác của System, trong đó hệ thống phải cung cấp các chỉ số thống kê giúp giáo viên đánh giá chất lượng đề thi và kết quả học tập. Đây là bài toán tổng hợp dữ liệu lớn và sinh báo cáo trực quan.

5.6.2 Giải pháp chi tiết

Hệ thống xây dựng module `analytics.service.js` cung cấp các API thống kê gồm: thống kê tổng quan (số lượng lớp, học sinh, đề thi, bài nộp); thống kê theo đề thi (điểm trung bình, độ lệch chuẩn, phổ điểm, tỉ lệ đạt); thống kê theo câu hỏi (độ khó, độ phân biệt, tỉ lệ chọn mỗi đáp án); và danh sách bài nộp gần đây.

Để tính độ khó và độ phân biệt của câu hỏi, hệ thống áp dụng các công thức cổ điển trong lý thuyết kiểm tra trắc nghiệm: độ khó p là tỉ lệ thí sinh trả lời đúng câu hỏi đó (giá trị càng gần 1 thì câu càng dễ); độ phân biệt D được tính theo phương pháp upper-lower groups, trong đó nhóm trên là 27% thí sinh có điểm cao nhất và nhóm dưới là 27% thí sinh có điểm thấp nhất; $D = p_{\text{trên}} - p_{\text{dưới}}$ với giá trị từ -1 đến 1, giá trị càng gần 1 thì câu càng có khả năng phân biệt tốt giữa học sinh giỏi và yếu.

Giao diện báo cáo sử dụng thư viện Recharts để hiển thị các biểu đồ: biểu đồ cột cho phổ điểm, biểu đồ tròn cho tỉ lệ đạt theo mức xếp loại (giỏi, khá, trung bình, yếu), biểu đồ đường cho xu hướng điểm theo thời gian, biểu đồ bảng nhiệt cho độ khó và độ phân biệt của từng câu hỏi. Hệ thống hỗ trợ xuất báo cáo ra PDF bằng thư viện jspdf và xuất ra Excel bằng thư việnxlsx.

5.6.3 Kết quả đạt được

Báo cáo thống kê cho phép giáo viên nhanh chóng xác định các câu hỏi có vấn đề (quá dễ, quá khó hoặc không phân biệt được) để điều chỉnh trong các lần ra đề sau. Thời gian sinh báo cáo cho một bài kiểm tra có khoảng 200 bài làm dưới 5 giây. Trong quá trình thử nghiệm, giáo viên đã sử dụng báo cáo để phát hiện và loại

bỏ 3 câu hỏi có độ phân biệt âm khởi ngân hàng câu hỏi.

5.7 Kết chương

Chương 5 đã trình bày chi tiết năm giải pháp kỹ thuật nổi bật và đóng góp chính của đề án, mỗi giải pháp đều được tổ chức theo cấu trúc ba mục: dẫn dắt bài toán, giải pháp chi tiết và kết quả đạt được. Năm giải pháp gồm:

- tự động sinh đề thi nhiều phiên bản và phiếu OMR kèm tọa độ nhận dạng;
- nhận dạng OMR trên thiết bị di động với khả năng phát hiện lỗi;
- tích hợp trợ lý AI đa nhà cung cấp;
- quản lý phiên người dùng theo vai trò;
- báo cáo và phân tích thống kê bài kiểm tra.

Các kết quả chính gồm: pipeline sinh đề-phiếu hoàn chỉnh với tọa độ chính xác dưới 1 pixel; thuật toán OMR trên di động đạt độ chính xác 96,2% trong điều kiện thực tế; khả năng phát hiện lỗi tô nhầm và tô nhẹ; tích hợp AI với cơ chế cache và chuyển đổi nhà cung cấp linh hoạt; và hệ thống báo cáo với các chỉ số cổ điển trong lý thuyết kiểm tra trắc nghiệm. Đây là phần cốt lõi đánh giá năng lực nghiên cứu và thực hành của sinh viên, đồng thời là cơ sở để đánh giá tính mới và tính khả thi của đề án trong Chương 6.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Tổng quan chương

Chương 5 đã trình bày chi tiết các giải pháp kỹ thuật và đóng góp nổi bật của đề án. Chương 6 - chương cuối cùng của đề án - tổng kết toàn bộ quá trình thực hiện, so sánh kết quả đạt được với các sản phẩm tương tự, phân tích những việc đã làm được và chưa làm được, đúc rút bài học kinh nghiệm và đề xuất hướng phát triển trong tương lai.

6.2 Kết luận

6.2.1 So sánh với các sản phẩm tương tự

Hệ thống Smart Grading hoàn thành đề án đã đáp ứng được đa số các tính năng quan trọng đã nêu trong bảng so sánh ở mục 2.2 và có một số điểm vượt trội so với các sản phẩm hiện có. So với phần mềm thương mại Remark Office OMR, hệ thống có ưu điểm là tích hợp đầy đủ quy trình quản lý đào tạo (lớp, học sinh, giáo viên) với quy trình chấm thi, hỗ trợ phúc khảo trực tuyến, có ứng dụng di động giúp giáo viên chấm bài mà không cần máy quét chuyên dụng và hoàn toàn miễn phí. So với Gradescope, hệ thống có ưu điểm là hỗ trợ tiếng Việt đầy đủ, có khả năng tạo nhiều mã đề tự động và tích hợp AI hỗ trợ sinh câu hỏi. So với công cụ mã nguồn mở OMRChecker, hệ thống có ưu điểm là có giao diện quản lý hoàn chỉnh, hỗ trợ phúc khảo và báo cáo thống kê.

Bên cạnh những ưu điểm, hệ thống cũng còn một số hạn chế so với các sản phẩm thương mại lâu năm như:

- chưa hỗ trợ phiếu trả lời có nhiều cột (multi-column) phức tạp như các kỳ thi chuẩn hóa quốc tế;
- chưa hỗ trợ chấm các dạng câu hỏi khác ngoài trắc nghiệm (ví dụ: điền vào chỗ trống, ghép đôi);
- độ chính xác của thuật toán OMR trên di động vẫn thấp hơn máy quét chuyên dụng khoảng 2-3%.

6.2.2 Những việc đã làm được

Trong suốt quá trình thực hiện đề án, sinh viên đã hoàn thành các công việc chính sau:

- khảo sát hiện trạng và phân tích yêu cầu chi tiết cho hệ thống chấm thi trắc nghiệm thông minh;
- thiết kế kiến trúc phần mềm theo mô hình phân lớp với ba tầng rõ ràng;

- xây dựng backend Node.js/Express với 17 model dữ liệu, 16 service nghiệp vụ và 17 nhóm API REST;
- xây dựng giao diện web React với hệ thống phân quyền theo vai trò, dashboard trực quan và đầy đủ các chức năng quản lý;
- xây dựng ứng dụng di động Flutter với module chấm OMR tích hợp OpenCV có độ chính xác 96,2%;
- xây dựng pipeline tự động sinh đề thi nhiều phiên bản kèm phiếu OMR thông qua gói AMC;
- tích hợp trợ lý AI đa nhà cung cấp (Gemini, OpenAI, Claude) để hỗ trợ sinh câu hỏi và phân tích báo cáo;
- triển khai thử nghiệm trong môi trường thực tế với 5 bài kiểm tra và khoảng 300 bài làm.

Các đóng góp nổi bật của đồ án gồm:

- xây dựng pipeline tự động sinh đề nhiều phiên bản và phiếu OMR kèm tọa độ nhận dạng - đây là điểm khác biệt quan trọng so với các giải pháp hiện có;
- triển khai thuật toán chấm OMR trên thiết bị di động với khả năng phát hiện lỗi tô nhầm, tô nhẹ và xử lý ảnh trong điều kiện thực tế;
- tích hợp AI trong cả hai chiều: hỗ trợ giáo viên tạo câu hỏi và hỗ trợ học sinh ôn tập thông qua trợ lý ảo.

6.2.3 Những việc chưa làm được

Bên cạnh những kết quả đạt được, đồ án vẫn còn một số hạn chế cần được khắc phục trong tương lai:

- hệ thống hiện chỉ hỗ trợ câu hỏi trắc nghiệm một đáp án và nhiều đáp án; chưa hỗ trợ các dạng câu hỏi khác như điền số, điền từ, ghép đôi;
- ứng dụng di động chưa hỗ trợ chế độ ngoại tuyến hoàn chỉnh; khi mất kết nối mạng trong thời gian dài, giáo viên vẫn cần đồng bộ thủ công khi có kết nối trở lại;
- hệ thống chưa hỗ trợ phiếu trả lời có hơn 4 đáp án cho mỗi câu hoặc các dạng phiếu phức tạp;
- chưa tích hợp thanh toán trực tuyến cho trường hợp hệ thống được triển khai thương mại;
- chưa có bản dịch sang tiếng Anh hoàn chỉnh cho giao diện, hạn chế khả năng mở rộng ra thị trường quốc tế.

6.2.4 Bài học kinh nghiệm

Qua quá trình thực hiện đồ án, sinh viên đã rút ra một số bài học kinh nghiệm quý báu. Thứ nhất, việc lựa chọn công nghệ phù hợp ngay từ đầu có ý nghĩa quyết định đến tiến độ và chất lượng đồ án; sinh viên đã mất khá nhiều thời gian để đánh giá các framework backend và frontend khác nhau trước khi quyết định. Thứ hai, việc thiết kế cơ sở dữ liệu cần được thực hiện cẩn thận ngay từ đầu vì việc thay đổi schema sau này rất tốn kém, đặc biệt là khi đã có dữ liệu thực tế. Thứ ba, quy trình kiểm thử cần được xây dựng song song với quá trình phát triển thay vì thực hiện ở cuối dự án; nếu kiểm thử sớm hơn, sinh viên có thể phát hiện và sửa lỗi nhanh hơn. Thứ tư, làm việc nhóm (dù chỉ một mình sinh viên thực hiện) cần có kế hoạch rõ ràng, sử dụng công cụ quản lý dự án và quản lý mã nguồn hiệu quả (Git, Jira) để tránh mất kiểm soát tiến độ.

6.3 Hướng phát triển

6.3.1 Hoàn thiện các chức năng hiện có

Trong thời gian tới, đồ án sẽ tập trung hoàn thiện các chức năng đã có để nâng cao chất lượng và trải nghiệm người dùng. Cụ thể,

- cải thiện độ chính xác của thuật toán OMR trên di động lên trên 98% bằng cách áp dụng các kỹ thuật học sâu (deep learning) để phát hiện ô tô, đặc biệt trong các trường hợp ảnh chụp ở điều kiện ánh sáng phức tạp;
- bổ sung chế độ ngoại tuyến hoàn chỉnh cho ứng dụng di động, cho phép giáo viên chấm bài ngay cả khi không có kết nối mạng và tự động đồng bộ khi có kết nối;
- bổ sung chức năng xuất báo cáo đa dạng hơn, cho phép tùy chỉnh nội dung và định dạng;
- tối ưu hóa hiệu năng của backend để đáp ứng được khối lượng truy cập lớn hơn trong các kỳ thi có hàng nghìn thí sinh.

6.3.2 Mở rộng tính năng mới

Bên cạnh hoàn thiện các tính năng hiện có, hệ thống có thể được mở rộng với các tính năng mới sau:

- hỗ trợ thêm các dạng câu hỏi khác ngoài trắc nghiệm như điền số, điền từ, ghép đôi và câu hỏi tự luận ngắn có chấm tự động bằng AI;
- hỗ trợ chấm bài thi trực tuyến kết hợp với bài thi trên giấy, cho phép giáo viên linh hoạt trong tổ chức thi;
- bổ sung chức năng tạo đề thi thông minh dựa trên phân tích ma trận kiến thức

và ngân hàng câu hỏi, đảm bảo đề thi có độ phủ và độ khó phù hợp;

- phát triển tính năng phân tích học tập cá nhân hóa cho học sinh dựa trên lịch sử làm bài, giúp học sinh nhận biết điểm mạnh - điểm yếu và có lộ trình ôn tập phù hợp;
- tích hợp thanh toán trực tuyến và quản lý gói dịch vụ để có thể triển khai hệ thống dưới dạng SaaS (Software as a Service);
- bổ sung bản dịch tiếng Anh cho toàn bộ giao diện để mở rộng thị trường;
- tích hợp với các hệ thống quản lý học tập (LMS) phổ biến như Moodle, Canvas thông qua chuẩn LTI (Learning Tools Interoperability).

6.3.3 Nâng cấp công nghệ

Về mặt công nghệ, một số hướng nâng cấp đáng chú ý gồm:

- chuyển đổi kiến trúc backend sang microservices khi quy mô hệ thống tăng lên, tách riêng các dịch vụ như sinh đề, chấm thi và phân tích để dễ dàng mở rộng độc lập;
- áp dụng công nghệ học sâu để nâng cao độ chính xác của thuật toán nhận dạng OMR và tự động phát hiện các trường hợp bất thường trong phiếu thi;
- tích hợp thêm các mô hình AI tiên tiến để nâng cao chất lượng phân tích báo cáo và trợ lý ảo cho học sinh;
- xây dựng hệ thống giám sát và cảnh báo sớm các vấn đề vận hành, giúp đội ngũ vận hành phản ứng nhanh với sự cố.

6.4 Kết chương

Chương 6 - chương cuối cùng của đồ án - đã tổng kết toàn bộ quá trình thực hiện và kết quả đạt được của hệ thống Smart Grading. Qua so sánh với các sản phẩm tương tự như Remark Office OMR, Gradescope và OMRChecker, hệ thống đã chứng minh được những điểm vượt trội về tính tích hợp, hỗ trợ tiếng Việt, khả năng tạo nhiều mã đề và chi phí triển khai thấp. Tám nhóm công việc chính đã hoàn thành từ khảo sát yêu cầu đến triển khai thử nghiệm đã được tổng kết, cùng với ba đóng góp nổi bật. Các hạn chế còn tồn tại được nhìn nhận thẳng thắn và bốn bài học kinh nghiệm quý báu đã được rút ra. Hướng phát triển được chia thành ba nhóm: hoàn thiện chức năng hiện có, mở rộng tính năng mới và nâng cấp công nghệ. Đồ án đã đạt được các mục tiêu đề ra và tạo nền tảng vững chắc cho các phiên bản tiếp theo của hệ thống, đồng thời cung cấp một giải pháp thực tiễn cho bài toán chấm thi trắc nghiệm tại các cơ sở giáo dục Việt Nam.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[23] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[24] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[25] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[26] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[27] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[28] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[29] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] J. L. Adams, “An introduction to optical mark recognition (omr) technology,” *Journal of Educational Technology Systems*, vol. 31, no. 3, pp. 261–276, 2003.
- [2] G. Bradski, “The opencv library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [4] OMRChecker, *Omrchecker: Omr sheet scanning & analysis*, <https://github.com/Udayraj123/OMRChecker>, Truy cập: 28-06-2026, 2022.
- [5] Auto Multiple Choice Project, *Auto multiple choice documentation*, <https://www.auto-multiple-choice.net/>, Truy cập: 28-06-2026, 2024.
- [6] H. Kopka and P. W. Daly, *A Guide to L^AT_EX: Document Preparation for Beginners and Advanced Users*, 4th ed. Pearson Addison-Wesley, 2004.
- [7] OpenJS Foundation, *Node.js documentation*, <https://nodejs.org/en/docs>, Truy cập: 28-06-2026, 2024.
- [8] Express.js Authors, *Express - fast, unopinionated, minimalist web framework for node.js*, <https://expressjs.com/>, Truy cập: 28-06-2026, 2024.
- [9] IETF, *Hypertext Transfer Protocol – HTTP/1.1*. Internet Engineering Task Force, 1999, RFC 2616.
- [10] MDN Web Docs, *Model-view-controller (mvc)*, <https://developer.mozilla.org/en-US/docs/Glossary/MVC>, Truy cập: 28-06-2026, 2024.
- [11] IETF, *JSON Web Token (JWT)*. Internet Engineering Task Force, 2015, RFC 7519.
- [12] Passport.js, *Passport.js - simple, unobtrusive authentication for node.js*, <https://www.passportjs.org/>, Truy cập: 28-06-2026, 2024.
- [13] MongoDB Inc., *Mongodb: The developer data platform*, <https://www.mongodb.com/docs>, Truy cập: 28-06-2026, 2024.
- [14] MongoDB Inc., *Mongoose odm documentation*, <https://mongoosejs.com/docs/>, Truy cập: 28-06-2026, 2024.
- [15] Meta Open Source, *React - a javascript library for building user interfaces*, <https://react.dev/>, Truy cập: 28-06-2026, 2024.
- [16] Poimandres, *Zustand - bear necessities for state management in react*, <https://github.com/pmndrs/zustand>, Truy cập: 28-06-2026, 2024.

- [17] Google, *Flutter documentation*, <https://docs.flutter.dev/>, Truy cập: 28-06-2026, 2024.
- [18] Felix Angelov, *Bloc pattern for flutter*, <https://bloclibrary.dev/>, Truy cập: 28-06-2026, 2024.
- [19] Google, *Gemini api documentation*, <https://ai.google.dev/docs>, Truy cập: 28-06-2026, 2024.
- [20] OpenAI, *Openai api reference*, <https://platform.openai.com/docs/api-reference>, Truy cập: 28-06-2026, 2024.
- [21] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [22] Cloudinary Ltd., *Cloudinary documentation*, <https://cloudinary.com/documentation>, Truy cập: 28-06-2026, 2024.
- [23] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [24] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [25] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [26] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [27] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [28] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [29] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>
- [30] Scott, James, *Software-defined networking*, <https://www.example.com/sdn>, 2013.
- [31] Ashton, Kevin, *That 'internet of things' thing*, <https://www.example.com/iot>, 2009.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

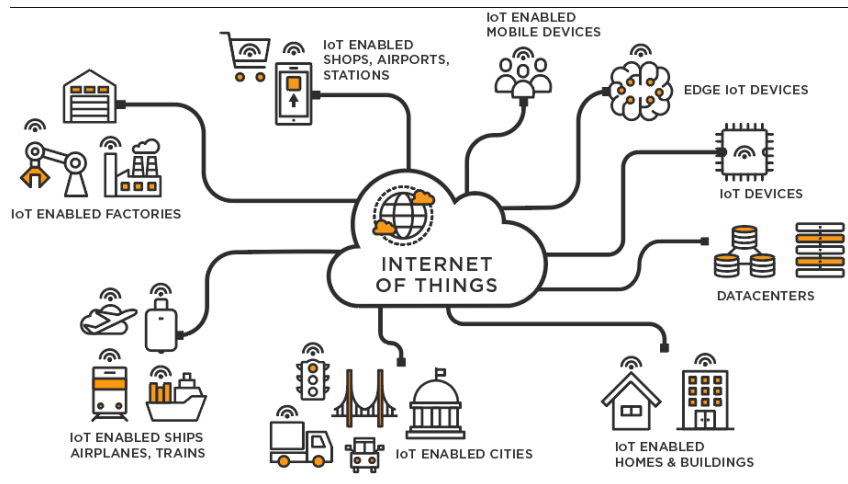
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [30]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [31]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

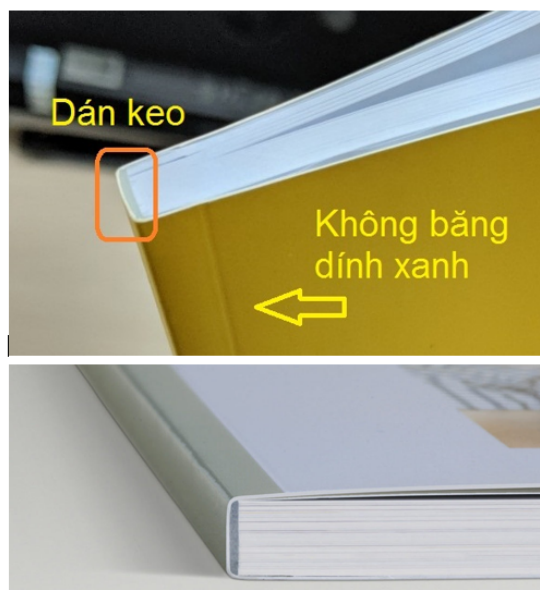
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

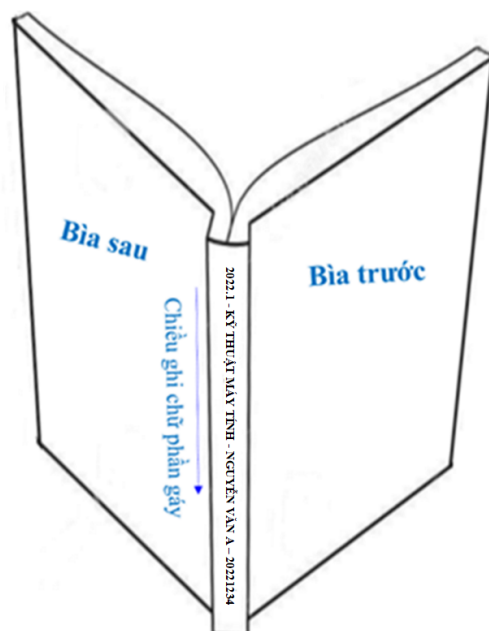
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Quy cách đóng quyển đồ án



Hình A.3: Quy cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Phần này trình bày đặc tả chi tiết cho các use case phụ của hệ thống Smart Grading không được đặc tả trong mục 2.3 của Chương 2 do hạn chế về dung lượng. Mỗi đặc tả gồm tên use case, luồng sự kiện chính, luồng sự kiện phát sinh, tiền điều kiện và hậu điều kiện, theo đúng mẫu đặc tả đã sử dụng trong Chương 2.

B.1 Đặc tả use case Quản lý trường học

Tên use case

Quản lý trường học.

Tiền điều kiện

Quản trị viên hệ thống đã đăng nhập với vai trò admin.

Luồng sự kiện chính

1. Quản trị viên truy cập chức năng "Quản lý trường học".
2. Hệ thống hiển thị danh sách các trường hiện có cùng các thông tin: tên trường, mã trường, địa chỉ, số lượng giáo viên, số lượng học sinh.
3. Quản trị viên chọn chức năng "Tạo trường mới".
4. Hệ thống hiển thị form tạo trường với các trường: tên trường, mã trường, địa chỉ, số điện thoại, email, cấu hình thang điểm.
5. Quản trị viên nhập thông tin và nhấn "Lưu".
6. Hệ thống kiểm tra tính hợp lệ của thông tin, lưu trường mới vào cơ sở dữ liệu và hiển thị thông báo thành công.
7. Quản trị viên có thể chọn một trường để xem chi tiết, cập nhật thông tin hoặc xóa trường.

Luồng sự kiện phát sinh

- Tại bước 6, nếu mã trường đã tồn tại: hệ thống thông báo và yêu cầu nhập mã khác.
- Tại bước 7, nếu trường đang có giáo viên hoặc học sinh hoạt động: hệ thống cảnh báo và yêu cầu xác nhận trước khi xóa.

Hậu điều kiện

Nếu thành công, trường mới được tạo và có thể được gán cho quản trị viên trường.

B.2 Đặc tả use case Quản lý giáo viên**Tên use case**

Quản lý giáo viên.

Tiền điều kiện

Quản trị viên hệ thống hoặc quản trị viên trường đã đăng nhập.

Luồng sự kiện chính

1. Người dùng truy cập chức năng "Quản lý giáo viên".
2. Hệ thống hiển thị danh sách giáo viên theo trường (nếu là school-admin) hoặc toàn hệ thống (nếu là admin).
3. Người dùng chọn "Thêm giáo viên".
4. Hệ thống hiển thị form thêm giáo viên với các trường: họ tên, email, mã giáo viên, môn phụ trách.
5. Người dùng nhập thông tin và nhấn "Lưu".
6. Hệ thống tạo tài khoản giáo viên, gửi email kích hoạt đến giáo viên.
7. Người dùng có thể chọn một giáo viên để xem chi tiết, cập nhật thông tin, phân công môn hoặc vô hiệu hóa tài khoản.

Luồng sự kiện phát sinh

- Tại bước 6, nếu email đã tồn tại: hệ thống thông báo và yêu cầu sử dụng email khác.
- Tại bước 7, nếu vô hiệu hóa tài khoản: hệ thống yêu cầu xác nhận vì giáo viên có thể đang phụ trách lớp đang hoạt động.

Hậu điều kiện

Nếu thành công, tài khoản giáo viên được tạo và có thể đăng nhập vào hệ thống sau khi kích hoạt qua email.

B.3 Đặc tả use case Quản lý học sinh**Tên use case**

Quản lý học sinh.

Tiền điều kiện

Quản trị viên trường hoặc giáo viên đã đăng nhập.

Luồng sự kiện chính

1. Người dùng truy cập chức năng "Quản lý học sinh".

2. Hệ thống hiển thị danh sách học sinh theo lớp hoặc theo trường.
3. Người dùng chọn "Thêm học sinh vào lớp".
4. Hệ thống cho phép thêm một học sinh mới (tạo tài khoản) hoặc thêm học sinh đã có tài khoản vào lớp.
5. Người dùng nhập thông tin cần thiết và nhấn "Lưu".
6. Hệ thống lưu thông tin và gửi thông báo cho học sinh.
7. Người dùng có thể chọn một học sinh để xem chi tiết, cập nhật thông tin, chuyển lớp hoặc xóa khỏi lớp.

Luồng sự kiện phát sinh

- Tại bước 5, nếu mã số sinh viên đã tồn tại: hệ thống thông báo và đề xuất sử dụng học sinh hiện có.
- Tại bước 7, nếu chuyển lớp: hệ thống yêu cầu chọn lớp đích và xác nhận.

Hậu điều kiện

Nếu thành công, học sinh được thêm vào lớp và có thể tham gia các bài kiểm tra của lớp.

B.4 Đặc tả use case Quản lý bài thi của học sinh

Tên use case

Quản lý các bài thi của học sinh.

Tiền điều kiện

Giáo viên đã đăng nhập, có bài làm đã được chấm tự động trên hệ thống.

Luồng sự kiện chính

1. Giáo viên truy cập chức năng "Bài nộp" và chọn bài kiểm tra tương ứng.
2. Hệ thống hiển thị danh sách bài làm của học sinh với các thông tin: tên học sinh, mã số, điểm, trạng thái.
3. Giáo viên chọn một bài làm để xem chi tiết.
4. Hệ thống hiển thị thông tin chi tiết: ảnh phiếu đã chấm, đáp án của học sinh, đáp án đúng, điểm từng câu, cảnh báo (nếu có).
5. Giáo viên có thể chỉnh sửa thủ công các câu bị nhận dạng sai bằng cách chọn lại đáp án đúng.
6. Giáo viên nhấn "Lưu chỉnh sửa" để cập nhật điểm.
7. Giáo viên có thể nhấn "Phê duyệt kết quả" để chuyển bài làm sang trạng thái

sẵn sàng công bố.

Luồng sự kiện phát sinh

- Tại bước 5, nếu giáo viên chỉnh sửa: hệ thống ghi nhận lịch sử chỉnh sửa và cập nhật điểm tự động.
- Tại bước 7, nếu bài làm chưa được phê duyệt: học sinh chưa thể xem điểm.

Hậu điều kiện

Nếu thành công, bài làm được cập nhật và sẵn sàng công bố cho học sinh khi giáo viên phát hành kết quả.

B.5 Đặc tả use case Xem bài thi và điểm số

Tên use case

Xem bài thi và điểm số.

Tiền điều kiện

Học sinh đã đăng nhập, bài thi đã có kết quả được công bố.

Luồng sự kiện chính

1. Học sinh truy cập chức năng "Bài thi của tôi" hoặc "Điểm của tôi".
2. Hệ thống hiển thị danh sách các bài thi đã tham gia với điểm tổng và trạng thái.
3. Học sinh chọn một bài thi để xem chi tiết.
4. Hệ thống hiển thị: điểm tổng, điểm từng câu, đáp án của học sinh, đáp án đúng, ảnh phiếu đã chấm.
5. Học sinh có thể xem các đơn phúc khảo đã gửi cho bài thi này và trạng thái của chúng.
6. Học sinh có thể tải về báo cáo điểm cá nhân dạng PDF.

Luồng sự kiện phát sinh

- Tại bước 4, nếu bài thi có câu bị giáo viên chỉnh sửa thủ công: hệ thống hiển thị thông báo để học sinh hiểu rõ.
- Tại bước 6, nếu tải về thất bại: hệ thống thông báo và cho phép thử lại.

Hậu điều kiện

Nếu thành công, học sinh nắm được kết quả chi tiết bài thi của mình.

B.6 Đặc tả use case Tạo câu hỏi, đề thi

Tên use case

Tạo câu hỏi, đề thi.

Tiền điều kiện

Giáo viên đã đăng nhập, có quyền quản lý ngân hàng câu hỏi.

Luồng sự kiện chính

1. Giáo viên truy cập chức năng "Ngân hàng câu hỏi".
2. Hệ thống hiển thị danh sách câu hỏi hiện có theo môn học và chủ đề.
3. Giáo viên chọn "Tạo câu hỏi mới".
4. Giáo viên chọn một trong ba phương thức: tạo thủ công, sinh bằng AI, hoặc nhập từ file.
5. Đối với tạo thủ công: hệ thống hiển thị form với các trường nội dung, loại câu hỏi, các đáp án, đáp án đúng, độ khó, chủ đề, giải thích.
6. Đối với sinh bằng AI: hệ thống hiển thị form yêu cầu nhập chủ đề và số lượng câu hỏi; giáo viên nhấn "Sinh câu hỏi"; hệ thống gọi dịch vụ AI và hiển thị kết quả; giáo viên có thể chỉnh sửa trước khi lưu.
7. Đối với nhập từ file: hệ thống cho phép tải lên file Excel/CSV theo mẫu.
8. Giáo viên lưu câu hỏi vào ngân hàng; câu hỏi được đánh dấu là "chờ phê duyệt" nếu do AI sinh.

Luồng sự kiện phát sinh

- Tại bước 6, nếu dịch vụ AI gặp lỗi: hệ thống thông báo và cho phép thử lại hoặc chuyển sang tạo thủ công.
- Tại bước 7, nếu file nhập không đúng định dạng: hệ thống thông báo lỗi cụ thể và cho phép tải mẫu.

Hậu điều kiện

Nếu thành công, câu hỏi được lưu vào ngân hàng và có thể sử dụng để tạo đề thi.

B.7 Đặc tả use case Đăng ký/Đăng nhập

Tên use case

Đăng ký/Đăng nhập.

Tiền điều kiện

Người dùng truy cập được vào ứng dụng web hoặc di động.

Luồng sự kiện chính

1. Người dùng truy cập trang đăng nhập.
2. Người dùng nhập email và mật khẩu, nhấn "Đăng nhập".
3. Hệ thống kiểm tra thông tin đăng nhập.
4. Nếu hợp lệ, hệ thống cấp access token và refresh token, chuyển hướng đến dashboard phù hợp với vai trò.
5. Nếu người dùng chưa có tài khoản, có thể chọn "Đăng ký" và làm theo quy trình đăng ký gồm nhập thông tin cá nhân, xác thực email.
6. Nếu người dùng quên mật khẩu, có thể chọn "Quên mật khẩu" và làm theo quy trình khôi phục qua email.

Luồng sự kiện phát sinh

- Tại bước 3, nếu thông tin sai: hệ thống thông báo lỗi và cho phép thử lại.
- Tại bước 4, nếu tài khoản chưa xác thực email: hệ thống yêu cầu xác thực trước.
- Tại bước 6, nếu email không tồn tại trong hệ thống: hệ thống thông báo và không gửi email khôi phục.

Hậu điều kiện

Nếu thành công, người dùng được đăng nhập và truy cập các chức năng theo vai trò.